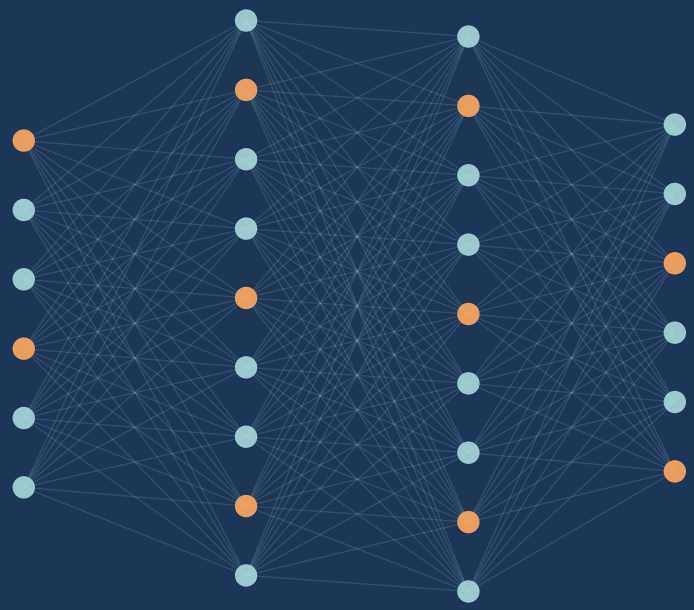
AI / ML ENGINEER

Advanced Interview Notes

The complete A–Z guide · Theory · Math · Systems · Code



Mathematics | Classical ML | Deep Learning | CNN · RNN | Transformers | LLMs · LoRA · RLHF | RAG · Agents | Generative AI | Reinforcement Learning | MLOps | System Design | Coding + Q&A

Prepared by

Made by Debasish Sahu

17 chapters · 30+ diagrams · tables · code recipes · 30+ interview Q&As · Edition 2026

Made by Debasish Sahu

Table of Contents

0. Interview Roadmap & How to Use These Notes . . . 5
0.1 What interviewers actually evaluate . . . 5
0.2 Topic weightage (typical, varies by company) . . . 5
0.3 8-week preparation plan . . . 6

1. Mathematical & Statistical Foundations . . . 7
1.1 Linear algebra essentials . . . 7
1.2 Probability & information theory . . . 7
1.3 Statistics & experimentation . . . 8
1.4 Optimisation theory . . . 8

2. Classical Machine Learning . . . 9
2.1 Learning paradigms . . . 9
2.2 Bias–variance trade-off & generalisation . . . 9
2.3 Linear & logistic models . . . 10
2.4 Tree-based models & ensembles . . . 10
2.5 SVM, kNN, Naive Bayes . . . 11
2.6 Unsupervised learning . . . 12
2.7 Recommender systems . . . 13
2.8 Algorithm cheat-comparison . . . 13

3. Model Evaluation, Data & Feature Engineering . . . 14
3.1 Classification metrics . . . 14
3.2 Regression, ranking & NLP metrics . . . 15
3.3 Validation strategy & data leakage . . . 15
3.4 Imbalanced data . . . 15
3.5 Feature engineering & preprocessing . . . 16
3.6 Interpretability & calibration . . . 16

4. Deep Learning Fundamentals . . . 17
4.1 Neural networks & backpropagation . . . 17
4.2 Activation functions . . . 17
4.3 Loss functions . . . 18
4.4 Initialisation & gradient problems . . . 18
4.5 Optimisers . . . 19
4.6 Regularisation & normalisation . . . 20
4.7 Practical training checklist . . . 21

5. Convolutional Networks & Computer Vision . . . 22
5.1 Convolution mechanics . . . 22
5.2 Landmark architectures . . . 22
5.3 Detection & segmentation . . . 23
5.3.1 Transfer learning recipe . . . 23

6. Sequence Models: RNN, LSTM, GRU . . . 24

7. Transformers — Deep Dive . . . 25

7.1 Architecture components & modern upgrades . . . 26
7.2 Parameter & compute accounting . . . 26
7.3 Tokenisation . . . 26

8. Large Language Models (LLMs) . . . 28
8.1 Pre-training & scaling laws . . . 28
8.2 Adaptation: prompting, fine-tuning, PEFT . . . 28
8.3 Alignment: RLHF, DPO, GRPO . . . 29
8.4 Decoding & sampling . . . 29
8.5 Efficient inference . . . 29
8.6 Evaluation, hallucination, safety . . . 30
8.7 Prompt engineering patterns . . . 30

9. RAG, Vector Search & AI Agents . . . 31
9.1 Building blocks of a strong RAG system . . . 31
9.2 AI agents . . . 32

10. Generative Models . . . 33
10.1 Diffusion in detail . . . 33

11. Reinforcement Learning . . . 35

12. MLOps & ML System Design . . . 36
12.1 MLOps building blocks . . . 36
12.2 Serving patterns & deployment strategies . . . 36
12.3 Monitoring & drift . . . 37
12.4 ML system-design framework (use in every design round) . . . 37
12.5 Worked design sketches . . . 37

13. Distributed Training & Performance Engineering . . . 38
13.1 Memory budgeting . . . 38
13.2 Performance checklist . . . 38

14. Coding Toolkit for ML Interviews . . . 40
14.1 Multi-head causal self-attention (PyTorch) . . . 40
14.2 Standard PyTorch training loop (AMP, clipping, scheduler) . . . 40
14.3 Logistic regression & k-means from scratch (NumPy) . . . 40
14.4 LoRA layer & ranking metrics . . . 41
14.5 SQL & pandas patterns frequently asked . . . 41
14.6 Algorithm & complexity cheat-sheet . . . 41

15. Interview Q&A; Bank . . . 42
15.1 Machine-learning fundamentals . . . 42
15.2 Deep learning . . . 42
15.3 Transformers & LLMs . . . 43
15.4 MLOps & system design . . . 43
15.5 Behavioural questions (STAR) . . . 44

16. Cheat Sheets & A–Z Glossary . . . 45
16.1 Formula sheet . . . 45
16.2 Default hyper-parameters (good starting points) . . . 45

16.3 A–Z Glossary 46
16.4 Final 24-hour checklist 46

0. Interview Roadmap & How to Use These Notes

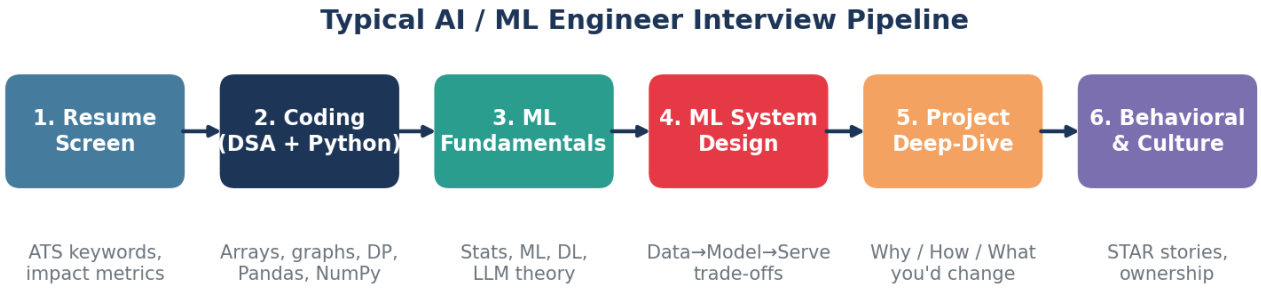


Figure 0.1 — A typical AI/ML Engineer hiring loop. Each round tests a different skill; prepare for all six.

0.1 What interviewers actually evaluate

Dimension	What they look for	How to show it
Fundamentals	Can you derive, not just recall? Bias-variance, MLE, backprop, attention	Explain *why*, write the equation, state assumptions and failure modes
Engineering	Clean, tested, efficient Python; NumPy/PyTorch fluency; SQL	Vectorise, mention complexity, handle edge cases, write small tests
Applied judgement	Choosing the right model/metric for the business problem	Start from the objective and constraints, then pick the simplest thing that works
System design	Data → training → serving → monitoring at scale	Draw the pipeline, quote latency/throughput numbers, discuss trade-offs
Research literacy	LLMs, RAG, agents, diffusion, RLHF — current state of the art	Compare approaches, know limitations, cite typical numbers
Communication	Structured, honest, concise	STAR stories; say 'I don't know, but I would test it by…'

0.2 Topic weightage (typical, varies by company)

Role type	ML theory	Deep learning / LLM	Coding	System design	MLOps
Applied ML Engineer	25%	20%	25%	20%	10%
LLM / GenAI Engineer	10%	35%	20%	25%	10%
Research Engineer	30%	40%	20%	5%	5%
ML Platform / MLOps	10%	10%	25%	30%	25%

0.3 8-week preparation plan

Week	Focus	Deliverable
1	Math: linear algebra, probability, statistics, optimisation (Ch.1)	One-page derivations: MLE for Gaussian/Bernoulli, PCA via SVD
2	Classical ML + evaluation (Ch.2–3)	Implement LR, k-means, decision tree from scratch
3	Deep learning core (Ch.4)	Train an MLP with your own backprop; compare optimisers
4	CNNs + sequence models (Ch.5–6)	Fine-tune a ResNet; explain LSTM gates on a whiteboard
5	Transformers + LLMs (Ch.7–8)	Write multi-head attention in PyTorch; LoRA fine-tune a small model
6	RAG, agents, generative models, RL (Ch.9–11)	Build a RAG app with hybrid search + reranker + eval
7	MLOps + system design (Ch.12–13)	Practise 4 design questions out loud with a timer
8	Mock interviews + Q&A bank (Ch.14–16)	Two full mock loops; refine STAR stories

INTERVIEW TIP Use the **ABCD answer framework**: **A**ssumptions → **B**asic solution → **C**omplexities / trade-offs → **D**eeper improvements. It keeps you structured under pressure and shows seniority.

KEY IDEA Answer every 'which model / metric?' question with: *objective → data → constraints (latency, cost, interpretability) → baseline → iterate*. Never jump straight to the fanciest model.

1. Mathematical & Statistical Foundations

1.1 Linear algebra essentials

- **Vector / matrix**: data point $x \in \mathbb{R}^d$; dataset $X \in \mathbb{R}^{n \times d}$; weights $W \in \mathbb{R}^{d_{out} \times d_{in}}$. A linear layer is a matrix multiply plus bias: $y = Wx + b$.
- **Norms**: $\|x\|_1 = \Sigma|x_i|$ (sparsity), $\|x\|_2 = \sqrt{\Sigma x_i^2}$ (energy), $\|x\|_\infty = \max|x_i|$. Frobenius norm $\|A\|_F = \sqrt{\Sigma\Sigma a_{ij}^2}$.
- **Rank** = number of independent columns. Low-rank structure underlies PCA, LoRA, matrix factorisation and attention approximations.
- **Eigen-decomposition**: $A v = \lambda v$. For symmetric A: $A = Q\Lambda Q^T$ with orthonormal Q. Covariance matrices are symmetric positive semi-definite (all $\lambda \geq 0$).
- **SVD**: any matrix $A = U\Sigma V^T$. Singular values σ_i measure importance of each direction; truncating to top-k gives the best rank-k approximation (Eckart–Young).
- **Positive definite** matrix: $x^TAx > 0\ \forall x \neq 0$ → convex quadratic, unique minimum. The Hessian being PSD ⇔ local convexity.

Matrix-calculus identity	Result
$\nabla_x(a^Tx)$	a
$\nabla_x(x^TAx)$	$(A + A^T)x$ (=2Ax if A symmetric)
$\nabla_w\|Xw - y\|^2$	$2X^T(Xw - y)$
$\partial\ \mathrm{tr}(AB)/\partial A$	B^T
$\nabla_W(a^TWb)$	ab^T

1.2 Probability & information theory

Bayes: $P(\theta \mid D) = P(D \mid \theta) \cdot P(\theta) / P(D)$ posterior ∝ likelihood × prior

Distribution	PMF / PDF	Mean	Var	Where it appears
Bernoulli(p)	$p^x(1-p)^{1-x}$	p	p(1−p)	Binary labels, logistic regression
Binomial(n,p)	$C(n,k)p^k(1-p)^{n-k}$	np	np(1−p)	Count of successes, A/B tests
Categorical(π)	$\Pi\ \pi_k^{1[x=k]}$	–	–	Softmax outputs, next-token prediction
Poisson(λ)	$\lambda^k e^{-\lambda}/k!$	λ	λ	Event counts, arrival rates
Gaussian(μ,σ²)	$(2\pi\sigma^2)^{-½}\exp(-(x-\mu)^2/2\sigma^2)$	μ	σ²	Noise, weight init, VAEs, CLT
Exponential(λ)	$\lambda e^{-\lambda x}$	1/λ	1/λ²	Waiting times, survival
Beta(α,β)	$x^{\alpha-1}(1-x)^{\beta-1}/B(\alpha,\beta)$	α/(α+β)	αβ/((α+β)²(α+β+1))	Prior for probabilities, Thompson sampling
Dirichlet(α)	$\Pi\ x_k^{\alpha_k-1}$	α_k/α_0	–	Topic models (LDA), priors on categoricals

- **Expectation** is linear: E[aX+bY]=aE[X]+bE[Y]. **Variance**: Var(X)=E[X²]−E[X]²; Var(X+Y)=Var X+Var Y+2Cov(X,Y).
- **Law of Large Numbers**: sample mean → true mean. **Central Limit Theorem**: mean of n i.i.d. samples ≈ N(μ, σ²/n) — the reason confidence intervals and z-tests work.
- **Independence** ⇒ uncorrelated, but not vice-versa (except for jointly Gaussian variables).
- **MLE**: θ = argmax Σ log $p(x_i|\theta)$. **MAP**: adds log-prior. Gaussian prior ⇒ L2 regularisation; Laplace prior ⇒ L1 regularisation.

Entropy $H(p) = -\Sigma\ p \log p$ Cross-entropy $H(p,q) = -\Sigma\ p \log q$ $KL(p\|q) = \Sigma\ p \log(p/q) = H(p,q) - H(p)$

- **KL divergence** ≥ 0, asymmetric. Minimising cross-entropy against fixed labels ≡ minimising KL ≡ maximising likelihood. Forward KL(p‖q) is *mean-seeking*; reverse KL(q‖p) is *mode-seeking* (used in VAEs/variational inference).

- **Mutual information** $I(X;Y)=KL(P(X,Y)\|P(X)P(Y))$ — used in feature selection and representation learning (InfoNCE / contrastive loss).
- **Perplexity** = exp(cross-entropy per token): the effective branching factor of a language model.

1.3 Statistics & experimentation

Concept	Definition	Interview note
p-value	P(data at least this extreme | H_0 true)	NOT the probability H_0 is true
Type I / II error	False positive (α) / false negative (β)	Power = 1−β; increase n or effect size to raise power
Confidence interval	Procedure covering true value in 95% of repeats	Not '95% chance the value lies here'
t-test / z-test	Compare means; t when σ unknown / small n	Check normality & independence; use Welch for unequal variance
Chi-square test	Independence between categorical variables	Expected counts ≥ 5 per cell
Multiple testing	Many tests inflate false positives	Bonferroni (α/m) or Benjamini–Hochberg (FDR)
Bootstrap	Resample with replacement to estimate uncertainty	Works for medians, AUC, any statistic
Sample size (A/B)	$n \approx 16\sigma^2/\delta^2$ per arm (α=.05, power=.8)	δ = minimum detectable effect

COMMON TRAP Peeking at an A/B test daily and stopping when $p < 0.05$ inflates the false-positive rate. Use a fixed horizon, sequential testing (SPRT, alpha-spending) or Bayesian methods.

1.4 Optimisation theory

- **Convex function**: $f(\lambda x+(1-\lambda)y) \le \lambda f(x)+(1-\lambda)f(y)$. Every local minimum is global. Linear/logistic regression and SVMs are convex; neural networks are **non-convex** but work in practice thanks to over-parameterisation, saddle-point escape and good initialisation.
- **Gradient descent**: $\theta \leftarrow \theta - \eta\nabla L(\theta)$. For an L-smooth convex function, $\eta \le 1/L$ guarantees convergence at rate $O(1/k)$. Strongly convex → linear rate.
- **Newton's method**: $\theta \leftarrow \theta - H^{-1}\nabla L$ — quadratic convergence but $O(d^3)$ per step; quasi-Newton (L-BFGS) approximates H.
- **Lagrange / KKT**: constrained optimisation min f(x) s.t. $g(x)\le 0$ → $L = f + \lambda g$; KKT conditions give SVM dual and its support vectors.
- **Stochastic gradients** are unbiased estimates with variance ∝ 1/batch size; noise helps escape sharp minima (better generalisation).

ADVANCED Hessian eigenvalues explain training dynamics: a large ratio $\lambda_{max}/\lambda_{min}$ (condition number) means narrow valleys and slow, oscillating SGD — the motivation for momentum, adaptive optimisers and normalisation layers.

2. Classical Machine Learning

2.1 Learning paradigms

Paradigm	Signal	Examples	Typical loss / objective
Supervised	Labelled (x,y)	Regression, classification, ranking	MSE, cross-entropy, hinge
Unsupervised	No labels	Clustering, density estimation, dimensionality reduction	Reconstruction error, likelihood
Self-supervised	Labels derived from data	LLM next-token, masked LM, contrastive (SimCLR, CLIP)	Cross-entropy, InfoNCE
Semi-supervised	Few labels + many unlabeled	Pseudo-labelling, consistency regularisation	Supervised + consistency
Reinforcement	Reward from interaction	Games, robotics, RLHF, bidding	Expected return

2.2 Bias–variance trade-off & generalisation

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2[\hat{f}(x)] + \text{Var}[\hat{f}(x)] + \sigma^2_{\text{noise}}$$

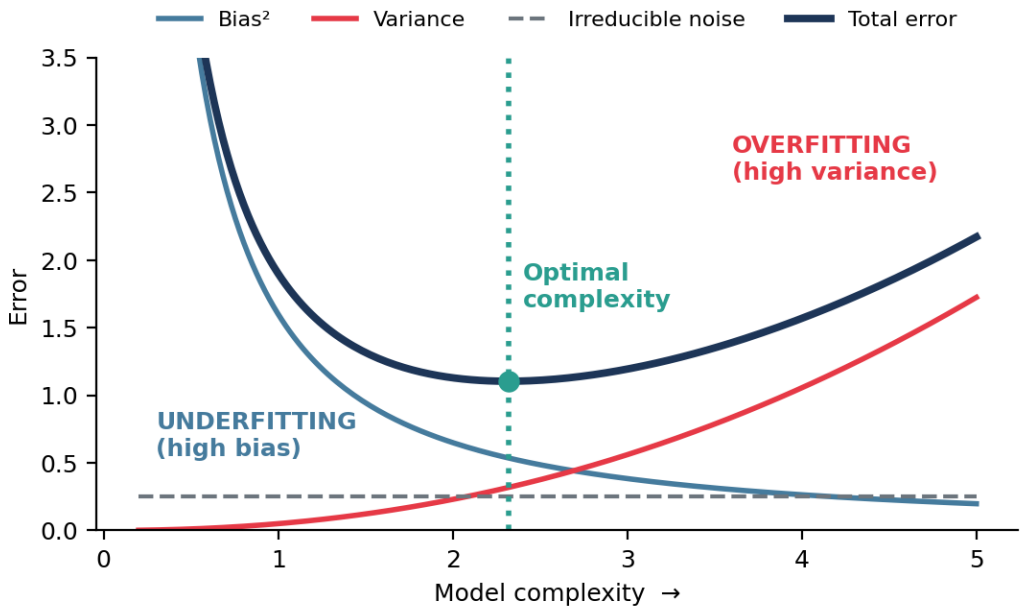


Figure 2.1 — Total error is minimised at the complexity where bias² and variance balance.

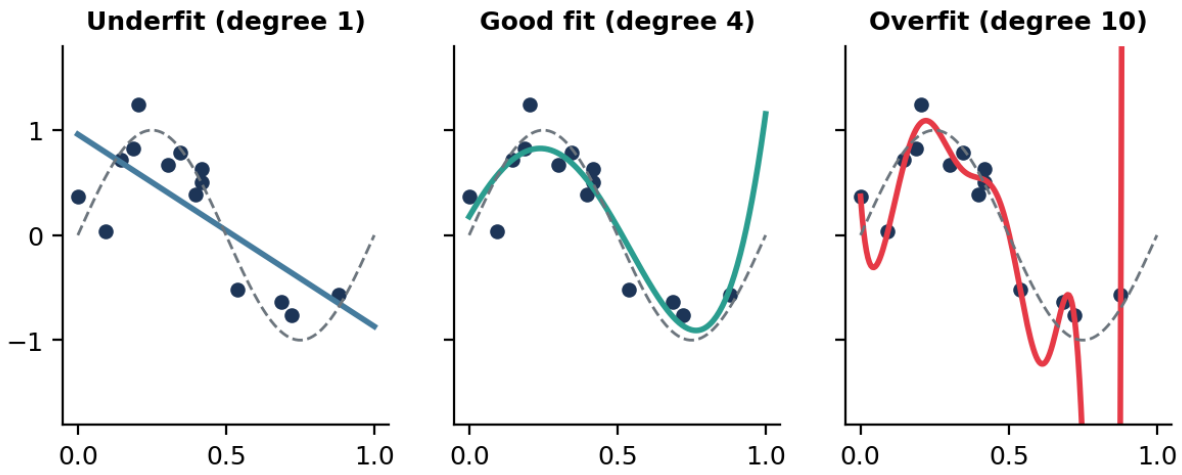


Figure 2.2 — The same data fitted with polynomials of degree 1, 4 and 13.

Symptom	Diagnosis	Fixes
Train error high, val error high	High bias (underfit)	Bigger model, better features, less regularisation, train longer
Train error low, val error high	High variance (overfit)	More data, regularisation, dropout, augmentation, early stopping, simpler model, ensembling
Both low but production poor	Distribution shift / leakage	Audit split, time-based validation, monitor drift

ADVANCED **Double descent**: in modern over-parameterised networks test error can fall again after the interpolation threshold — classical U-shape is not the whole story. Implicit regularisation of SGD and large-scale data explain why huge models generalise.

2.3 Linear & logistic models

- **Linear regression**: $\hat{y} = w^T x + b$. Closed form (normal equation) $\mathbf{w = (X^T X)^{-1} X^T y}$; equals MLE under Gaussian noise. Assumptions: linearity, independent errors, homoscedasticity, no multicollinearity (else $X^T X$ is near-singular → use Ridge).
- **Logistic regression**: $p(y=1|x)=\sigma(w^T x+b)$, $\sigma(z)=1/(1+e^{-z})$. Loss (log-loss / BCE): $-[y \log p + (1-y) \log(1-p)]$. Gradient is elegantly $\mathbf{X^T(p - y)/n}$ — same form as linear regression.
- **Softmax regression**: $p_k=e^{z_k}/\Sigma e^{z_j}$; gradient of CE w.r.t. logits = p − y (one-hot).
- **GLMs** generalise: Gaussian→identity link, Bernoulli→logit, Poisson→log. Pick the family matching your target.

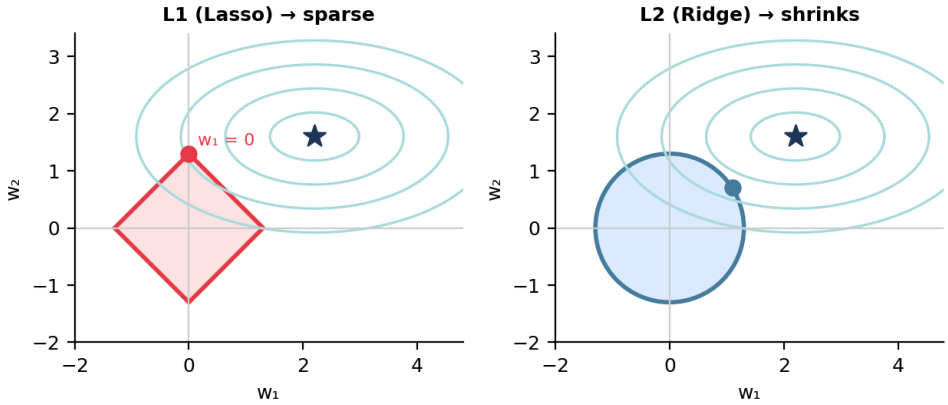


Figure 2.3 — Loss contours meet the constraint region: the L1 diamond's corners produce exact zeros (sparsity); the L2 ball only shrinks.

Regulariser	Penalty	Effect	Bayesian view
L2 / Ridge	$\lambda\|w\|_2^2$	Shrinks all weights, handles collinearity, closed form	Gaussian prior
L1 / Lasso	$\lambda\|w\|_1$	Sparse solution → feature selection	Laplace prior
Elastic Net	$\lambda_1\|w\|_1 + \lambda_2\|w\|_2^2$	Sparsity + stability for correlated features	Mixed
Dropout / weight decay	Implicit / decoupled	Standard in deep nets (AdamW decouples decay)	Approx. model averaging

2.4 Tree-based models & ensembles

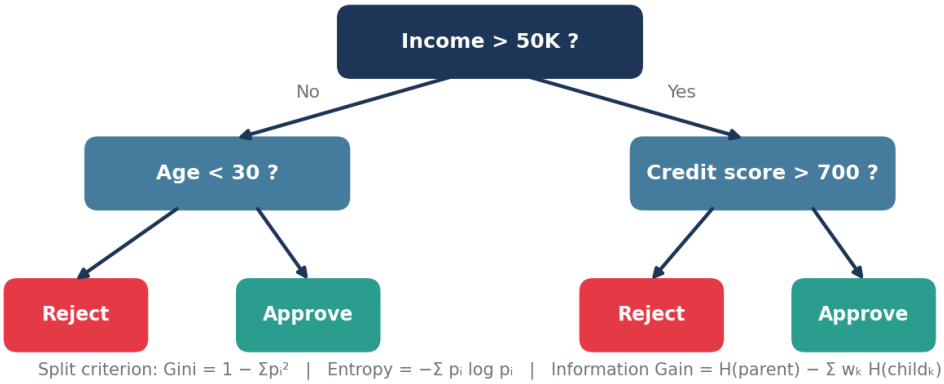


Figure 2.4 — A decision tree recursively partitions feature space using the split with the largest impurity reduction.

- **Decision tree**: greedy top-down splits maximising information gain (or Gini decrease). Handles mixed types and non-linearities, invariant to monotonic scaling, but high variance → prune, limit depth, or ensemble.
- **Random Forest** = bagging + random feature subsets per split. Reduces variance; out-of-bag (OOB) error gives free validation; feature importance via impurity or permutation.
- **Gradient Boosting**: fit each new tree to the negative gradient (pseudo-residual) of the loss: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$. Reduces bias; needs shrinkage η, subsampling and early stopping to avoid overfitting.

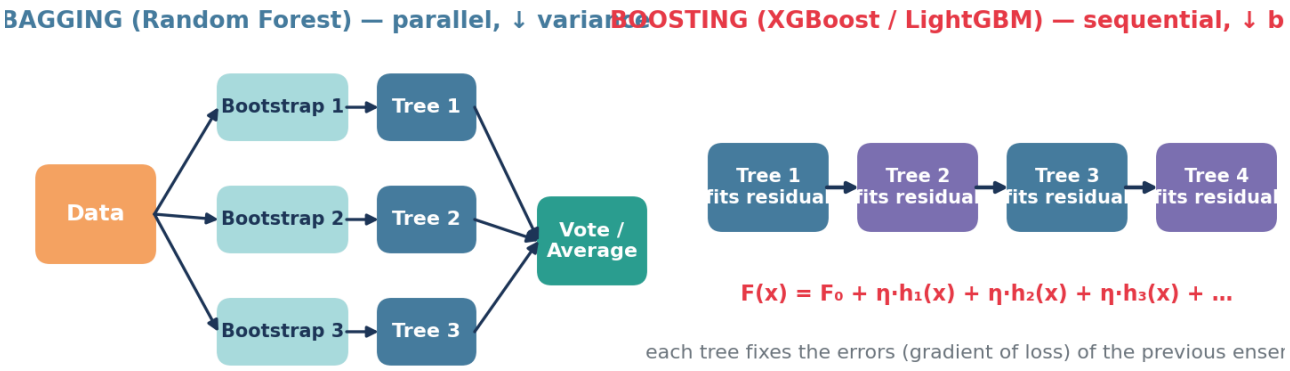


Figure 2.5 — Bagging trains independent trees in parallel; boosting trains trees sequentially on residuals.

Library	Key ideas	Best for
XGBoost	2nd-order Taylor of loss (gradient *g* and Hessian *h*); leaf weight w* = −G/(H+λ); regularised objective; sparsity-aware splits	Strong general baseline for tabular data
LightGBM	Histogram binning, leaf-wise growth, GOSS sampling, EFB feature bundling	Very large datasets, speed
CatBoost	Ordered boosting + ordered target statistics to avoid target leakage; symmetric trees	Many high-cardinality categoricals

INTERVIEW TIP For tabular data, well-tuned gradient-boosted trees usually beat deep learning. Say this confidently, then explain the exceptions: very large data, multimodal inputs, or when embeddings must be learned end-to-end.

2.5 SVM, kNN, Naive Bayes

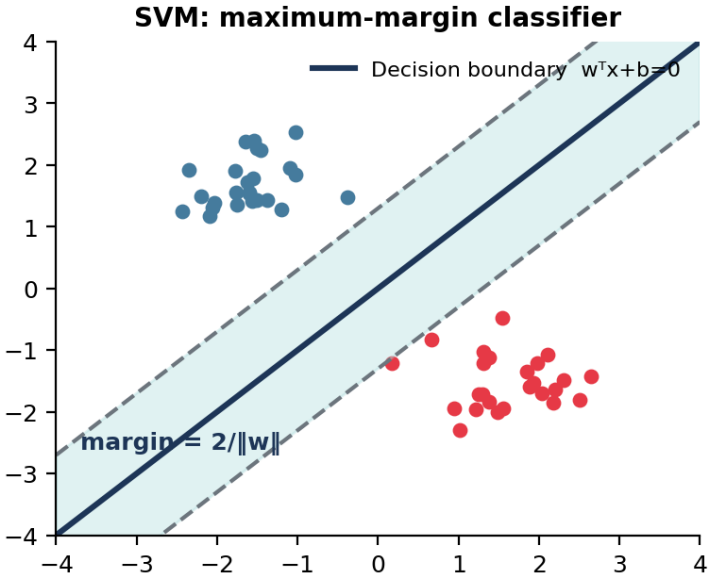


Figure 2.6 — SVM finds the hyperplane with maximum margin 2/‖w‖; only support vectors define it.

- **SVM**: min ½‖w‖² + C Σ max(0, 1 − y_i($w^T x_i$+b)) (hinge loss). **Kernel trick**: replace $x^T x'$ with k(x,x′) (RBF: exp(−γ‖x−x′‖²)) to get non-linear boundaries without explicit features. Large C → less regularisation. Scales poorly (O(n²–n³)).
- **k-NN**: non-parametric, lazy; prediction = majority/mean of k nearest neighbours. Needs feature scaling; suffers from the curse of dimensionality (distances concentrate). Use ANN indices (HNSW, FAISS) at scale.
- **Naive Bayes**: P(y|x) ∝ P(y)ΠP(x_i|y) assuming conditional independence. Surprisingly strong for text; fast; probabilities are poorly calibrated.

2.6 Unsupervised learning

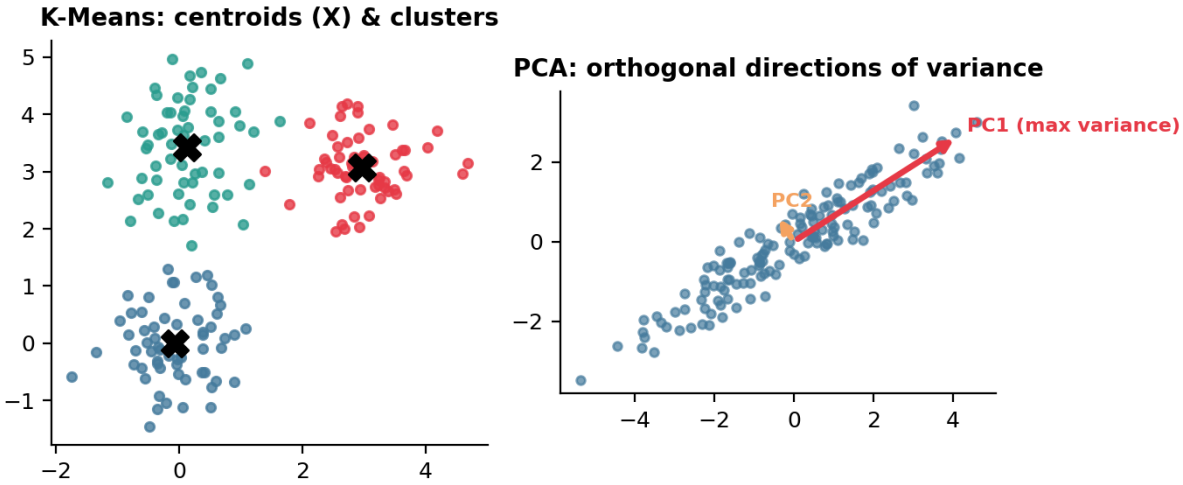


Figure 2.7 — Left: k-means partitions data around centroids. Right: PCA finds orthogonal directions of maximal variance.

- **k-means**: alternate assign → update centroids to minimise within-cluster SS. Sensitive to init (use k-means++), assumes spherical clusters, choose k via elbow / silhouette / gap statistic.
- **GMM + EM**: soft clustering; E-step computes responsibilities, M-step re-estimates μ, Σ, π. Generalises k-means (covariance = σ²I, σ→0).
- **DBSCAN**: density-based, finds arbitrary shapes and noise; parameters ε and minPts. **Hierarchical**: agglomerative with linkage (Ward, average).
- **PCA**: centre data, eigen-decompose covariance (or SVD of X); project on top-k components; explained variance ratio = λ_k/Σλ. Linear. **t-SNE / UMAP**: non-linear, for visualisation — distances between clusters are not meaningful.
- **Anomaly detection**: Isolation Forest, One-Class SVM, autoencoder reconstruction error, Mahalanobis distance.

2.7 Recommender systems

Approach	Idea	Pros / cons
Content-based	Match item features to user profile	No cold-start for items; limited serendipity
Collaborative filtering	Users with similar history like similar items	Powerful; cold-start & sparsity problems
Matrix factorisation	$R \approx UV^T$ minimising $\Sigma(r-u\cdot v)^2 + \lambda(\|U\|^2+\|V\|^2)$ (ALS/SGD)	Scalable; implicit feedback via weighted MF
Two-tower retrieval	User and item encoders; dot-product; ANN search	Millisecond retrieval over millions of items
Ranking (LTR)	GBDT / DNN with pointwise, pairwise (RankNet), listwise (LambdaMART) loss	Optimises NDCG directly

KEY IDEA Industrial recommenders are **multi-stage**: candidate generation (recall, $\sim 10^3$ from 10^7) → ranking (precision) → re-ranking (diversity, business rules). Discuss exploration (bandits) and position bias.

2.8 Algorithm cheat-comparison

Algorithm	Scaling needed	Handles non-linearity	Interpretable	Training speed	Typical weakness
Linear / Logistic	Yes	No (needs features)	High	Very fast	Underfits complex data
Decision tree	No	Yes	High	Fast	Overfits, unstable
Random Forest	No	Yes	Medium	Medium	Large memory, slow inference
Gradient boosting	No	Yes	Medium	Medium	Many hyper-params, overfit risk
SVM (RBF)	Yes	Yes	Low	Slow on big n	Poor scaling
k-NN	Yes	Yes	Medium	None (lazy)	Slow inference, high-d
Neural network	Yes	Yes	Low	Slow	Data-hungry, tuning heavy

3. Model Evaluation, Data & Feature Engineering

3.1 Classification metrics

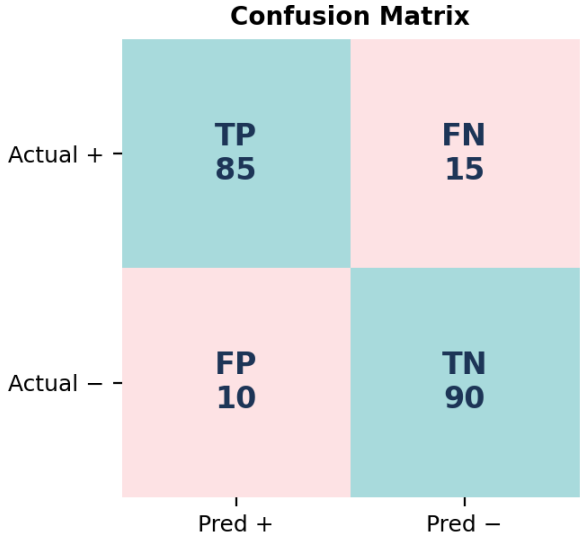


Figure 3.1 — Confusion matrix: every classification metric is derived from these four counts.

Metric	Formula	Use when
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Balanced classes only
Precision	TP/(TP+FP)	False positives are costly (spam filter, recommendations)
Recall / TPR	TP/(TP+FN)	False negatives are costly (cancer, fraud, safety)
Specificity	TN/(TN+FP)	Need low false-alarm rate
F1 / F_β	2PR/(P+R) ; (1+β²)PR/(β²P+R)	Imbalanced data; β>1 weights recall
ROC-AUC	P(score(pos) > score(neg))	Ranking quality, threshold-free, balanced-ish data
PR-AUC / AP	Area under precision–recall	Heavily imbalanced data (rare positives)
Log-loss	−mean[y log p + (1−y) log(1−p)]	Probabilistic quality; penalises confident errors
MCC	(TP·TN−FP·FN)/√((TP+FP)(TP+FN)(TN+FP)(TN+FN))	Single balanced score for imbalance
Brier / ECE	Mean squared prob. error / calibration gap	Are predicted probabilities trustworthy?

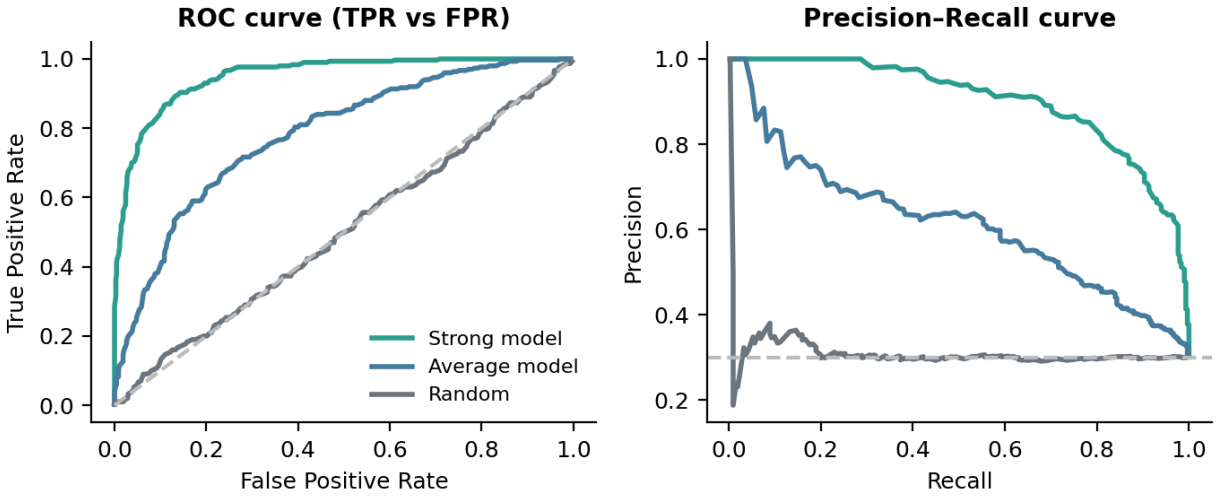


Figure 3.2 — ROC and PR curves. With 30% positives the random-PR baseline is 0.3; ROC baseline is the diagonal.

COMMON TRAP ROC-AUC can look great on a 1:1000 imbalanced problem while precision is terrible. Report PR-AUC and precision@k for rare-event tasks, and pick the threshold from business cost, not 0.5.

3.2 Regression, ranking & NLP metrics

Task	Metrics	Notes
Regression	MSE/RMSE, MAE, R^2, MAPE, Huber, quantile (pinball) loss	MAE robust to outliers; RMSE punishes large errors; MAPE breaks at $y \approx 0$
Ranking / retrieval	Recall@k, Precision@k, MRR, MAP, NDCG@k	NDCG = DCG/IDCG with DCG = $\Sigma (2^{rel}-1)/\log_2(i+1)$
Language modelling	Perplexity, bits-per-byte	Only comparable with the same tokeniser
Translation / summarisation	BLEU (n-gram precision), ROUGE (recall), BERTScore, COMET	Weak correlation with human judgement; add human/LLM eval
Generation quality	FID, Inception Score, CLIP score	Distribution-level, need many samples
Object detection	IoU, AP@[.5:.95], mAP	$IoU = \lvert A \cap B \rvert / \lvert A \cup B \rvert$

3.3 Validation strategy & data leakage

- **Hold-out** (train/val/test) for large data; **k-fold CV** for small data; **stratified** k-fold keeps class ratios; **group k-fold** when samples of one entity (user, patient) must not straddle splits; **time-series split** (expanding window) for temporal data — never shuffle.
- **Nested CV**: inner loop tunes hyper-parameters, outer loop estimates generalisation — avoids optimistic bias.
- **Leakage** = information at training that will not exist at prediction time: target-derived features, future data, fitting scalers/encoders on the full dataset, duplicates across splits, post-event features. Always fit preprocessing *inside* the training fold (use sklearn Pipeline).

INTERVIEW TIP When a model shows suspiciously good validation metrics, your first hypothesis should be **leakage**, not brilliance. Inspect top feature importances for anything that 'knows the future'.

3.4 Imbalanced data

Technique	How it works	Caveat
Class weights / cost-sensitive loss	Multiply loss of minority class	Simple first choice; keep probabilities recalibrated
Resampling	Over-sample minority (SMOTE interpolates neighbours) / under-sample majority	Resample **only** the training fold; SMOTE can create noisy samples
Focal loss	$-\alpha(1-p_t)^{\gamma} \log p_t$; down-weights easy examples	Popular in detection (RetinaNet)
Threshold moving	Choose operating point on PR curve	Free; do it before anything fancy
Anomaly framing	One-class / reconstruction models	When positives are extremely rare

3.5 Feature engineering & preprocessing

Topic	Practices
Scaling	Standardise (z-score) for linear/SVM/NN/kNN; min-max for bounded inputs; trees do not need it. Log-transform heavy tails.
Categorical encoding	One-hot (low cardinality), ordinal (ordered), target/mean encoding with K-fold + smoothing (avoid leakage), hashing trick, learned embeddings (high cardinality).
Missing values	Impute median/mode/model-based; add 'is_missing' indicator; understand MCAR / MAR / MNAR; trees like XGBoost handle NaN natively.
Outliers	Winsorise/clip, robust scalers, or use robust losses (Huber); never blindly delete signal.
Text / time / geo	TF-IDF, embeddings; cyclical encoding sin/cos of hour/day; lag & rolling-window features; geohash / distance features.
Feature selection	Filter (mutual information, variance), wrapper (RFE), embedded (L1, tree importance), permutation importance, SHAP.
Dimensionality	PCA / autoencoders; remove highly correlated features (VIF > 10 indicates multicollinearity).

3.6 Interpretability & calibration

- **SHAP**: Shapley-value attributions — additive, consistent; TreeSHAP is exact and fast for trees. **LIME**: local linear surrogate. **PDP / ICE** plots show marginal effects; **permutation importance** measures drop in score.
- **Calibration**: predicted 0.8 should be right 80% of the time. Fix with Platt scaling (sigmoid), isotonic regression, or temperature scaling for neural nets; visualise with reliability diagrams.

4. Deep Learning Fundamentals

4.1 Neural networks & backpropagation

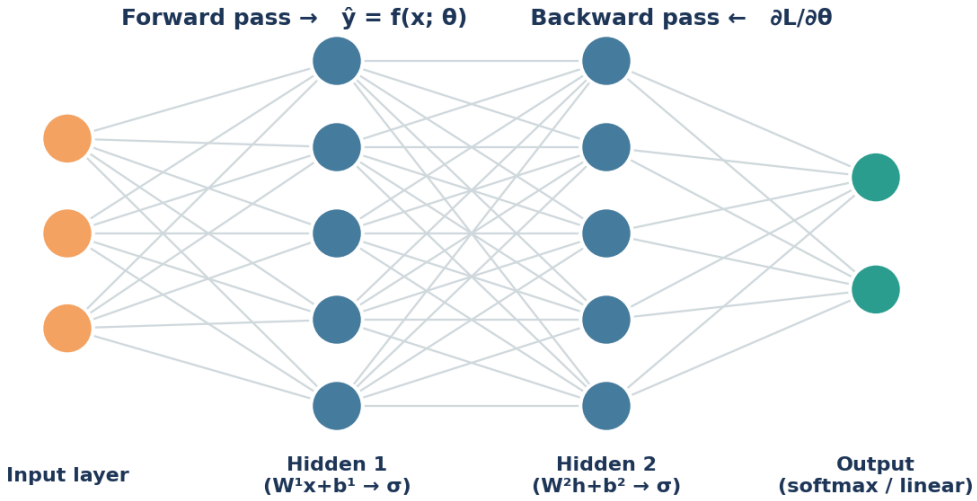


Figure 4.1 — A multilayer perceptron: each layer is an affine map followed by a non-linearity.

$$h^{(l)} = \sigma(\, W^{(l)}\, h^{(l-1)} + b^{(l)}\,), \qquad \hat{y} = h^{(L)}, \qquad \theta \leftarrow \theta - \eta\, \nabla_{\theta}\, L(\hat{y}, y)$$

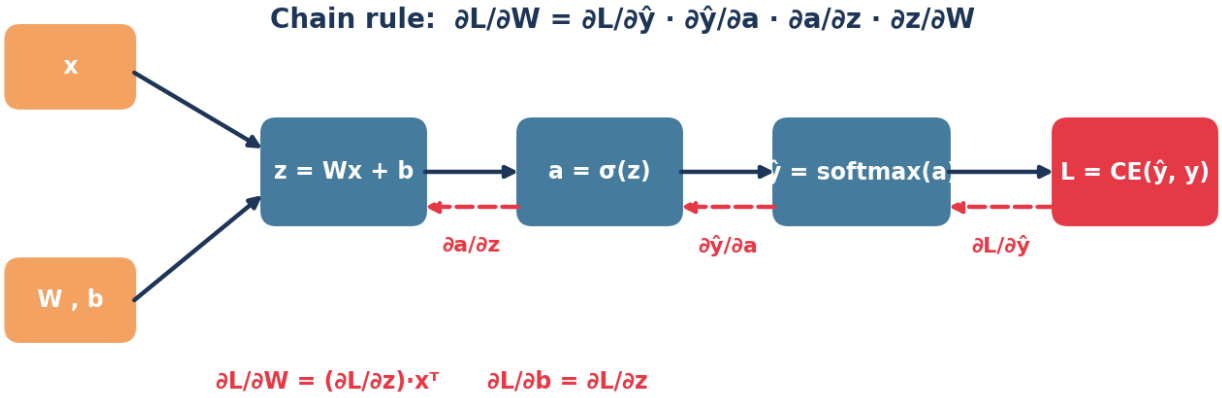


Figure 4.2 — Backpropagation applies the chain rule through the computational graph in reverse.

- **Universal approximation**: a wide enough single hidden layer can approximate any continuous function on a compact set — but depth gives exponentially more efficient representations.
- **Backprop** = reverse-mode automatic differentiation: one forward pass caches activations, one backward pass computes all gradients at cost ≈ 2–3× the forward pass. Memory is dominated by stored activations (→ gradient checkpointing trades compute for memory).
- **Autograd** frameworks (PyTorch, JAX) build a dynamic graph; *loss.backward()* accumulates gradients — remember *optimizer.zero_grad()*.
- **Shape discipline**: for a batch X ($B \times d_{in}$) and layer W ($d_{in} \times d_{out}$), $dL/dW = X^{T} \cdot (dL/dY)$, $dL/dX = (dL/dY) \cdot W^{T}$.

4.2 Activation functions

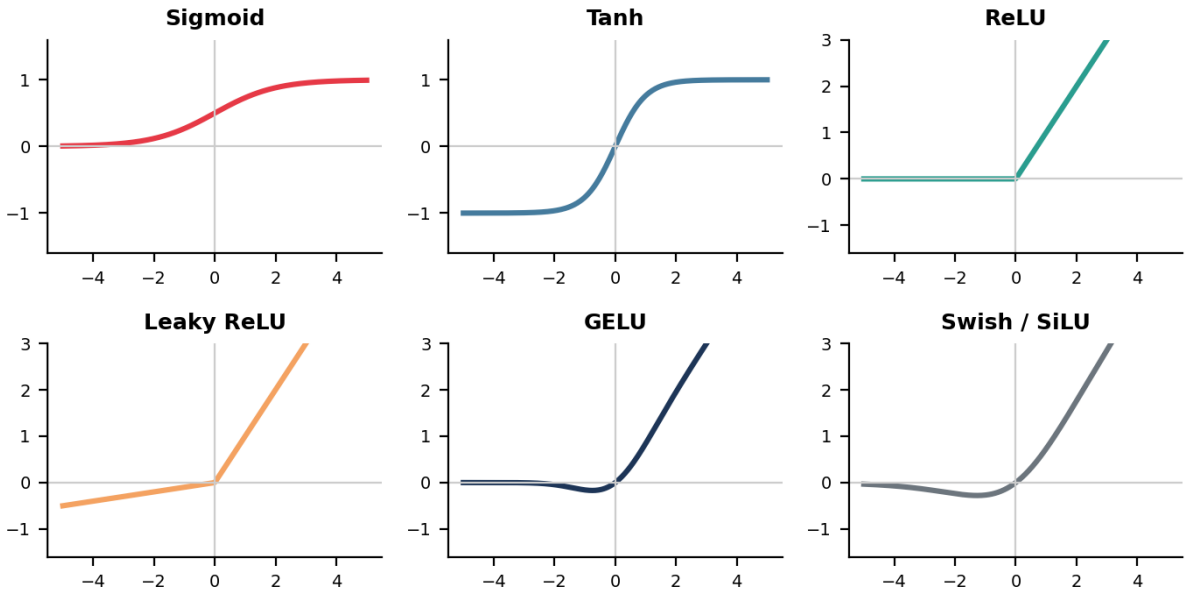


Figure 4.3 — Common activation functions.

Function	Formula	Pros	Cons / use
Sigmoid	$1/(1+e^{-x})$	Probability output	Saturates → vanishing gradient; not zero-centred. Use only for binary outputs / gates
Tanh	$(e^{x}-e^{-x})/(e^{x}+e^{-x})$	Zero-centred	Still saturates. LSTM candidate values
ReLU	max(0,x)	Cheap, sparse, no saturation for x>0	'Dying ReLU' for x<0. Default for CNN/MLP
Leaky / PReLU	max(αx, x)	Fixes dying units	Extra hyper-parameter
GELU	x·Φ(x)	Smooth; used in BERT/GPT	Slightly costlier
SwiGLU	Swish(xW)⊙(xV)	Best quality in modern LLMs (LLaMA, PaLM)	Needs extra projection (hidden ≈ 8/3·d)
Softmax	$e^{z_i}/\Sigma e^{z_j}$	Categorical distribution	Subtract max(z) for numerical stability

4.3 Loss functions

Task	Loss	Formula / note
Regression	MSE / MAE / Huber	Huber = quadratic near 0, linear in tails → robust
Binary classification	Binary cross-entropy	Use logits + BCEWithLogits for numerical stability
Multi-class	Categorical cross-entropy	$-\Sigma\ y_k \log p_k$; label smoothing ε improves calibration
Metric learning	Contrastive / Triplet / InfoNCE	InfoNCE = $-\log(\exp(s^{+}/\tau)/\Sigma\exp(s/\tau))$
Segmentation	Dice / IoU / Focal	Handles pixel-level imbalance
Generative	Adversarial, ELBO, denoising MSE	See Chapter 10
Ranking	Pairwise hinge / BPR / ListNet	Optimise order not values

4.4 Initialisation & gradient problems

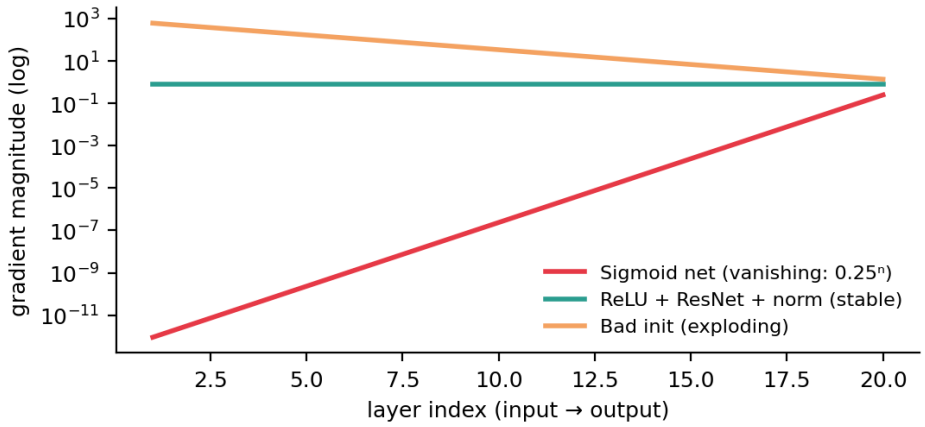


Figure 4.4 — Gradient magnitude across depth: vanishing (sigmoid) vs stable (ReLU + residual + normalisation) vs exploding.

Init	Variance	For
Xavier / Glorot	$2/(n_{in}+n_{out})$	tanh / sigmoid
He / Kaiming	$2/n_{in}$	ReLU family
Small normal (0.02)	fixed	Transformers (GPT-2 style), with residual-scaled output layers

- **Vanishing gradients**: product of many derivatives < 1. Remedies: ReLU/GELU, proper init, residual connections, normalisation, LSTM/GRU gating.
- **Exploding gradients**: product > 1 (RNNs especially). Remedies: **gradient clipping** (clip global norm to 1.0), smaller LR, careful init.
- **Dead ReLUs / loss spikes**: lower LR, warm-up, check data (NaNs, bad labels), use bf16 rather than fp16, add z-loss.

4.5 Optimisers

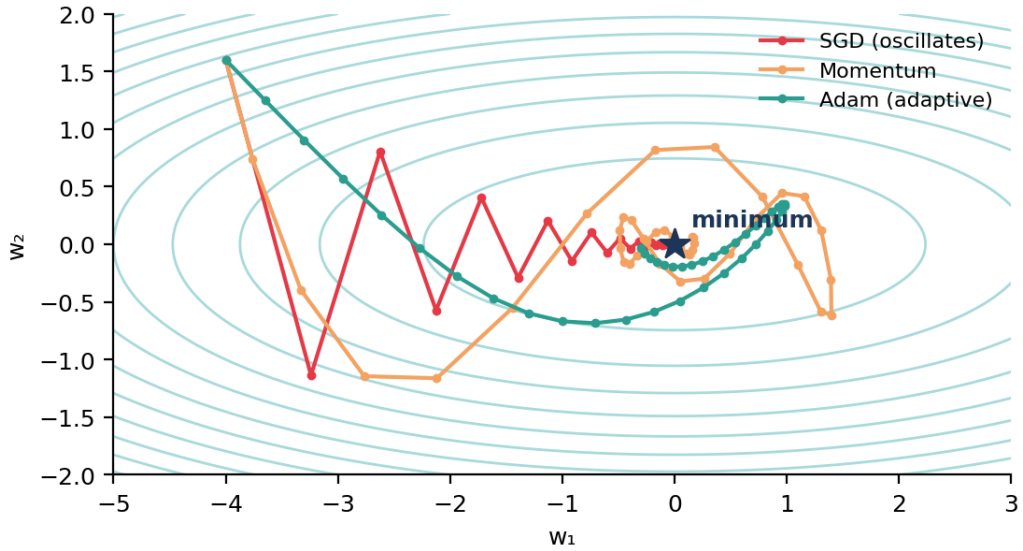


Figure 4.5 — On an ill-conditioned valley, SGD zig-zags, momentum accelerates, Adam adapts per-parameter step sizes.

Optimiser	Update rule	Notes
SGD + Momentum	$v \leftarrow \beta v + g$; $\theta \leftarrow \theta - \eta v$	Great generalisation with tuning; standard for CNNs
RMSProp	$s \leftarrow \rho s + (1-\rho)g^2$; $\theta \leftarrow \theta - \eta g/(\sqrt{s}+\varepsilon)$	Per-parameter scaling
Adam	$m \leftarrow \beta_1 m+(1-\beta_1)g$; $v \leftarrow \beta_2 v+(1-\beta_2)g^2$; $\theta \leftarrow \theta - \eta \cdot \hat{m}/(\sqrt{\hat{v}}+\varepsilon)$	β_1=0.9, β_2=0.999 (0.95 for LLMs), ε=1e-8; bias-corrected

Optimiser	Update rule	Notes
AdamW	Adam + **decoupled** weight decay $\theta \leftarrow \theta - \eta\lambda\theta$	Default for Transformers; L2 inside Adam is not equal to weight decay
Adafactor / Lion / Sophia	Memory-lean or sign/second-order variants	Used for huge models

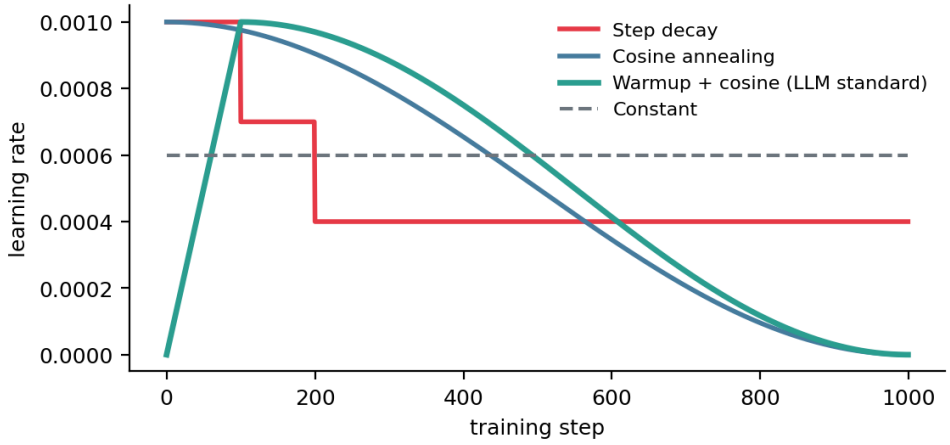


Figure 4.6 — Learning-rate schedules; warm-up + cosine decay is the LLM default.

INTERVIEW TIP Learning rate is the single most important hyper-parameter. Use an LR range test, warm-up for the first 0.5–2% of steps, and scale LR roughly ∝ √(batch size) (or linearly with warm-up) when you change batch size.

4.6 Regularisation & normalisation

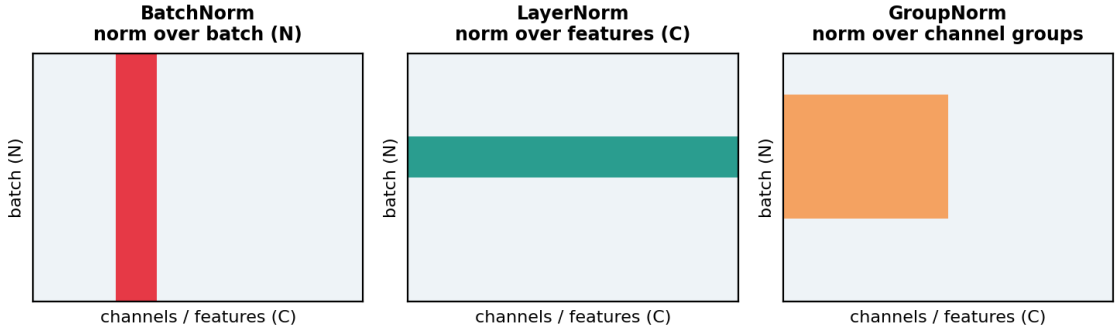


Figure 4.7 — Which axes each normalisation layer averages over (red/green/gold cells).

Technique	What it does	Notes
BatchNorm	Normalise over batch per channel; learn γ, β; running stats at inference	Accelerates CNNs; fails with tiny batches; train/eval behaviour differs
LayerNorm / RMSNorm	Normalise across features per token	Standard in Transformers; RMSNorm drops mean-centering (faster)
GroupNorm	Normalise groups of channels	Good for small batches (detection/segmentation)
Dropout	Randomly zero activations with prob. p (scale 1/(1−p))	Off at inference; ≈ ensemble; rarely used in large LLM pretraining
Weight decay / L2	Penalise large weights	AdamW λ ≈ 0.01–0.1
Data augmentation	Flip, crop, MixUp, CutMix, RandAugment, SpecAugment	Cheapest way to add ‘data’
Early stopping	Stop when validation loss stops improving	Keep best checkpoint
Label smoothing	Soft targets (1−ε, ε/(K−1))	Prevents over-confidence

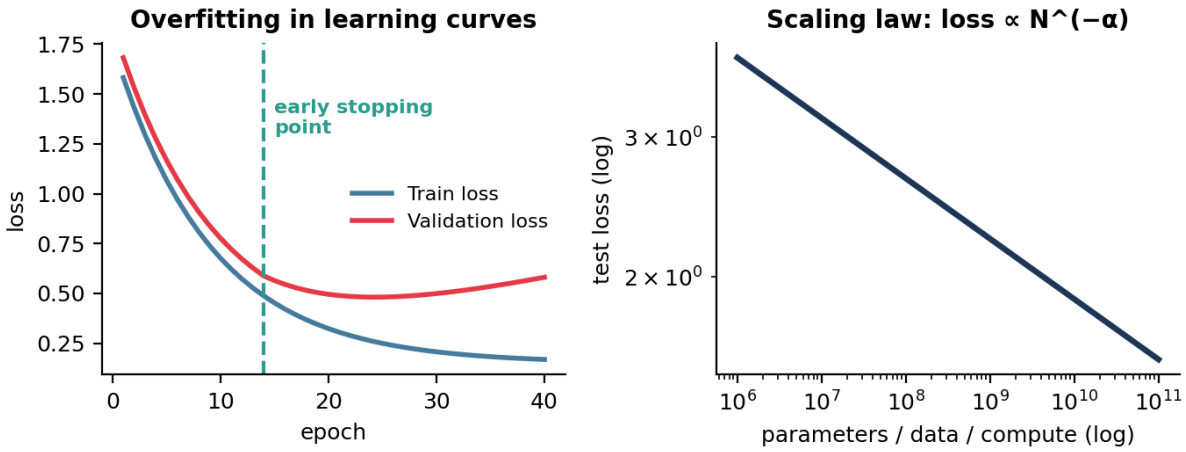


Figure 4.8 — Left: learning curves reveal overfitting and the early-stopping point. Right: empirical scaling law (power-law).

4.7 Practical training checklist

- **Overfit a tiny batch first** (loss → ~0) to prove the model and pipeline can learn.
- Sanity-check the initial loss: for K classes it should be ≈ ln K.
- Log train/val loss, gradient norm, LR, activation statistics, throughput; watch for NaNs.
- Mixed precision (bf16 preferred over fp16 — no loss scaling), gradient accumulation for large effective batch, EMA of weights, seed everything, checkpoint often.
- Change one thing at a time; keep an experiment tracker (Weights & Biases, MLflow).

5. Convolutional Networks & Computer Vision

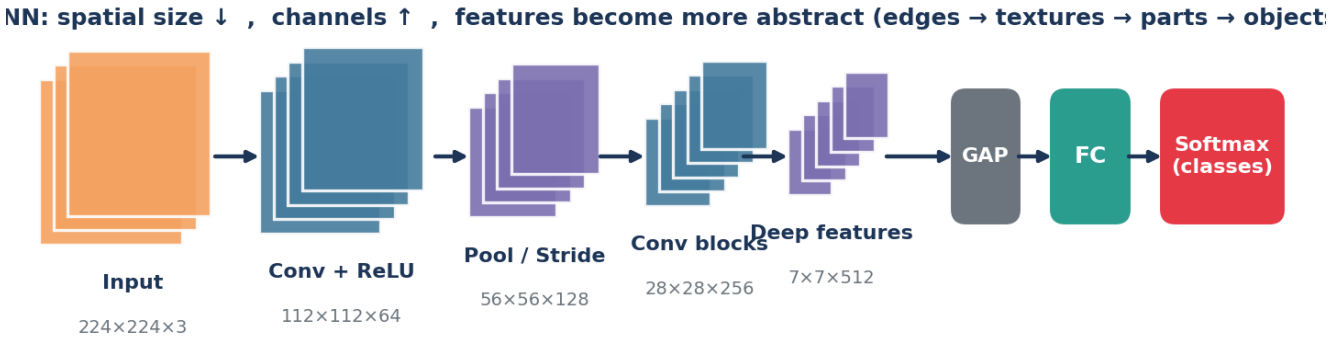


Figure 5.1 — Typical CNN: spatial resolution shrinks while channel depth grows.

5.1 Convolution mechanics

$$\text{Output size } O = \lfloor (N + 2P - K) / S \rfloor + 1 \qquad \text{Params (conv)} = (K \cdot K \cdot C_{in} + 1) \cdot C_{out}$$

- **Inductive biases**: locality, weight sharing, translation equivariance — far fewer parameters than a fully-connected layer on pixels.
- **Receptive field** grows with depth, stride and dilation. **Pooling** gives small translation invariance; **global average pooling** replaces big FC heads.
- **1×1 convolution** mixes channels cheaply (bottlenecks); **depthwise-separable** conv (MobileNet) cuts cost ≈ 8–9×; **dilated** conv enlarges receptive field without pooling; **transposed conv** upsamples.
- Worked example: 3×3 conv, 64→128 channels has (9·64+1)·128 = 73,856 parameters.

5.2 Landmark architectures

Model	Year	Key idea	Impact
LeNet / AlexNet	1998 / 2012	ReLU, dropout, GPU training	Started deep-learning era
VGG	2014	Stacks of 3×3 convs	Simplicity; heavy FC layers
Inception / GoogLeNet	2014	Parallel multi-scale branches, 1×1 bottleneck	Efficient width
ResNet	2015	**Skip connections** y = F(x) + x	Enabled 100+ layers; gradient highway
DenseNet	2016	Concatenate all earlier feature maps	Feature reuse
MobileNet / EfficientNet	2017–19	Depthwise-separable convs; compound scaling (depth, width, res)	Edge deployment
ViT / Swin	2020–21	Split image into patches, Transformer encoder; shifted windows	Beats CNNs with large data; foundation of CLIP
ConvNeXt	2022	Modernised CNN (large kernels, LN, GELU)	CNNs remain competitive

5.3 Detection & segmentation

Task	Models	Concepts
Object detection	Faster R-CNN (two-stage), YOLO / SSD / RetinaNet (one-stage), DETR (Transformer, set prediction)	Anchors, IoU, Non-Maximum Suppression, mAP, focal loss, FPN for multi-scale
Semantic segmentation	FCN, U-Net (encoder-decoder + skip), DeepLab (atrous convs)	Per-pixel CE + Dice; mIoU
Instance segmentation	Mask R-CNN, SAM (promptable)	Detect then segment each object
Self-supervised vision	SimCLR, MoCo, DINO, MAE	Contrastive or masked-patch prediction; no labels
Vision-language	CLIP, SigLIP, LLaVA, Flamingo	Align image & text embeddings with contrastive loss

5.3.1 Transfer learning recipe

- Small data: freeze backbone, train new head → unfreeze top blocks with 10× smaller LR (discriminative LRs).
- Match preprocessing (mean/std, resolution) to pretraining. Fine-tune with strong augmentation; monitor for catastrophic forgetting.
- Deployment: quantisation (INT8), pruning, distillation, ONNX/TensorRT, channels-last layout.

INTERVIEW TIP Be ready to explain **why ResNet works**: the identity shortcut lets gradients flow unimpeded ($\partial y/\partial x = \partial F/\partial x + 1$), and the block only has to learn a residual — easier to optimise than a full mapping.

6. Sequence Models: RNN, LSTM, GRU

- **RNN**: $h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$. Shares weights across time; processes variable-length input but is sequential (no parallelism) and forgets long-range context.
- **Backprop Through Time (BPTT)**: unroll the network; gradient contains $\Pi\, \partial h_t/\partial h_{t-1}$ → vanishes ($\|W\|<1$) or explodes ($\|W\|>1$). Truncated BPTT + gradient clipping is standard.

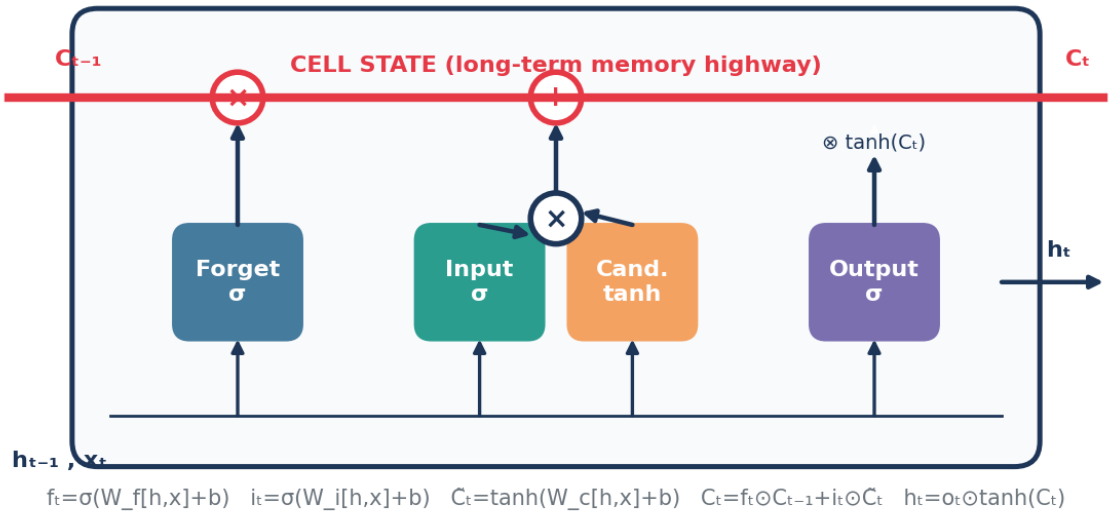


Figure 6.1 — LSTM cell: three gates regulate a protected cell-state 'memory highway'.

Gate	Role	Equation
Forget f_t	What to erase from memory	$\sigma(W_f[h_{t-1},x_t]+b_f)$
Input i_t	What new info to write	$\sigma(W_i[h_{t-1},x_t]+b_i)$
Output o_t	What to expose as h_t	$\sigma(W_o[h_{t-1},x_t]+b_o)$
Cell update	Additive path → stable gradients	$C_t = f_t \odot C_{t-1} + i_t \odot \tanh(W_c[\cdot]+b_c)$

- **GRU**: merges cell and hidden state, two gates (update z, reset r) → ~25% fewer parameters, similar accuracy.
- **Bidirectional RNN** reads both directions (for tagging, not for generation). **Seq2Seq**: encoder compresses input to a vector, decoder generates output; the fixed-size bottleneck motivated **attention** (Bahdanau, 2014) which lets the decoder look back at all encoder states — the seed of the Transformer.
- **Why Transformers replaced RNNs**: parallel training over the sequence, O(1) path length between any two tokens, better scaling. RNN-like models return in linear-time forms: **state-space models (S4, Mamba)** and RWKV, offering O(n) inference with constant memory.

Aspect	RNN/LSTM	Transformer	SSM (Mamba)
Training parallelism	Sequential	Fully parallel	Parallel scan
Long-range dependency	Weak	Strong (global attention)	Strong, compressed state
Compute per token (inference)	O(1)	O(n) with KV-cache	O(1)
Memory over context	O(1)	O(n)	O(1)

7. Transformers — Deep Dive

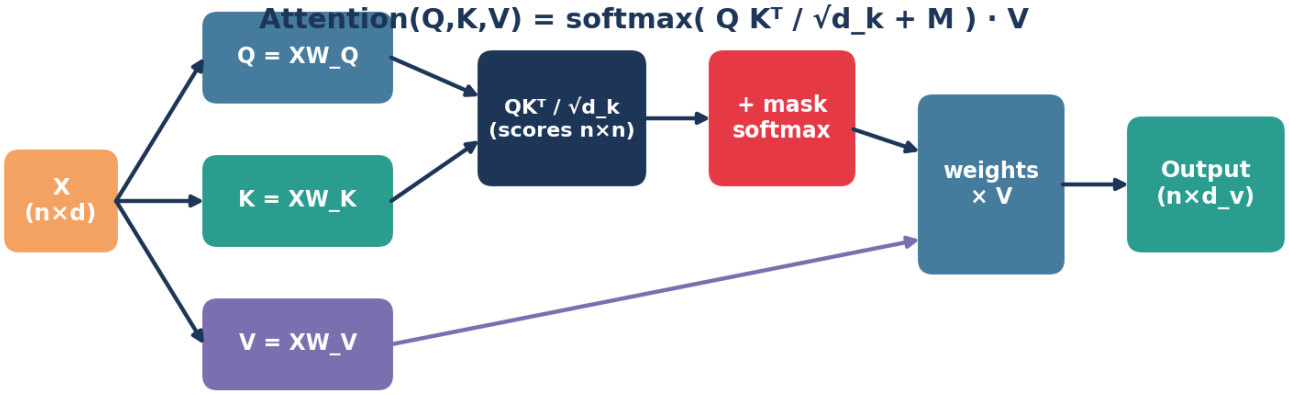


Figure 7.1 — Scaled dot-product attention.

$$\text{Attention}(Q,K,V) = \text{softmax}(\, Q\, K^{T} / \sqrt{d_k} + M \,)\, V \qquad \text{MultiHead} = \text{Concat}(\text{head}_1 \ldots \text{head}_h)\, W^{O}$$

- **Intuition**: each token issues a **query**; every token offers a **key** (what I contain) and a **value** (what I give). Similarity(query,key) → softmax weights → weighted sum of values.
- **Why divide by $\sqrt{d_k}$**: dot products of d_k-dim random vectors have variance d_k; without scaling, softmax saturates and gradients vanish.
- **Multi-head**: h parallel subspaces ($d_k = d/h$) let different heads specialise (syntax, coreference, position…). Cost is the same as one big head.
- **Causal mask M** ($-\infty$ above the diagonal) stops a token seeing the future — required for autoregressive language models.
- **Complexity**: time $O(n^2 \cdot d)$, memory $O(n^2)$ for the score matrix — the bottleneck for long contexts. Feed-forward layers cost $O(n \cdot d^2)$.

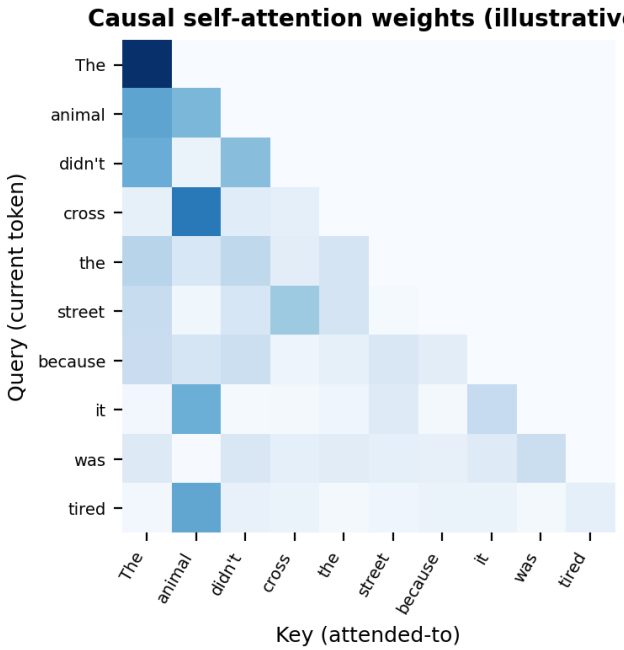


Figure 7.2 — Attention weights let 'it' attend strongly to 'animal' (illustrative).

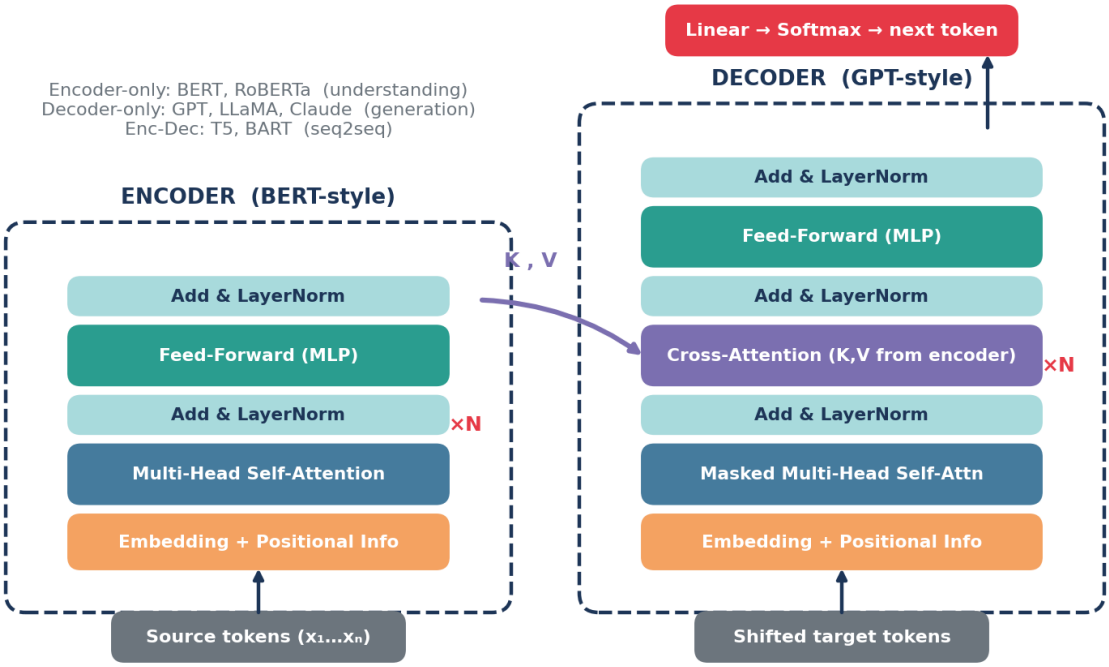


Figure 7.3 — Encoder and decoder Transformer stacks. Modern LLMs use decoder-only.

7.1 Architecture components & modern upgrades

Component	Original (2017)	Modern LLM practice	Why
Positional info	Sinusoidal absolute	**RoPE** (rotate q,k by position angle); ALiBi	Relative position, better length extrapolation
Normalisation	Post-LN LayerNorm	**Pre-LN + RMSNorm**	Stable training of deep stacks; cheaper
FFN activation	ReLU, hidden 4d	**SwiGLU**, hidden ≈ 8d/3	Higher quality per FLOP
Attention heads	Multi-Head (MHA)	**GQA / MQA**: share K,V across head groups	Shrinks KV-cache 4–8×, faster decoding
Attention kernel	Naive n×n	**FlashAttention** (tiling in SRAM, exact)	O(n) memory, 2–4× faster, IO-aware
Bias terms	Yes	Removed	Simplicity, stability
Sparsity	Dense	**Mixture-of-Experts** (top-k routing)	More parameters at constant FLOPs
Context	512–2k	128k–1M+ (RoPE scaling, YaRN, ring attention)	Long documents, agents

7.2 Parameter & compute accounting

Params per layer ≈ $12\ d^2$ ($4d^2$ attention + $8d^2$ FFN) Total ≈ $12 \cdot L \cdot d^2 + V \cdot d$

- GPT-3: L=96, d=12,288 → $12 \cdot 96 \cdot 12288^2 \approx 174B$ (+ embeddings) ≈ 175B.
- **Training FLOPs ≈ 6·N·D** (N params, D tokens); **inference ≈ 2·N FLOPs per token**.
- **KV-cache per token** = $2 \cdot L \cdot n_{kv_heads} \cdot d_{head} \cdot$ bytes. LLaMA-2-70B (80 layers, 8 KV heads, d_{head}=128, fp16) ≈ 320 KB/token → 4k context ≈ 1.3 GB per sequence.
- **Mixture-of-Experts**: total parameters ≫ active parameters (Mixtral-8×7B ≈ 47B total, ≈ 13B active). Needs a load-balancing loss and expert-parallel communication.

7.3 Tokenisation

- **BPE / byte-level BPE** merges frequent byte pairs; **WordPiece** (BERT) maximises likelihood; **SentencePiece / Unigram** is language-agnostic. Vocabulary sizes 32k–256k.

- Trade-off: larger vocab → shorter sequences but bigger embedding/softmax and rarer tokens. Tokenisation explains odd behaviour on arithmetic, spelling, rare languages and code whitespace.
- Embedding matrix is often **tied** with the output projection to save parameters.

Family	Objective	Examples	Best at
Encoder-only	Masked LM (predict masked tokens)	BERT, RoBERTa, DeBERTa	Classification, retrieval embeddings, NER
Decoder-only	Next-token prediction (causal)	GPT, LLaMA, Claude, Mistral	Open-ended generation, in-context learning
Encoder-decoder	Span corruption / seq2seq	T5, BART, Whisper	Translation, summarisation, speech

INTERVIEW TIP Classic question: *'Why does a Transformer need positional encoding?'* — Self-attention is permutation-equivariant; without position signals the model sees a bag of tokens.

8. Large Language Models (LLMs)

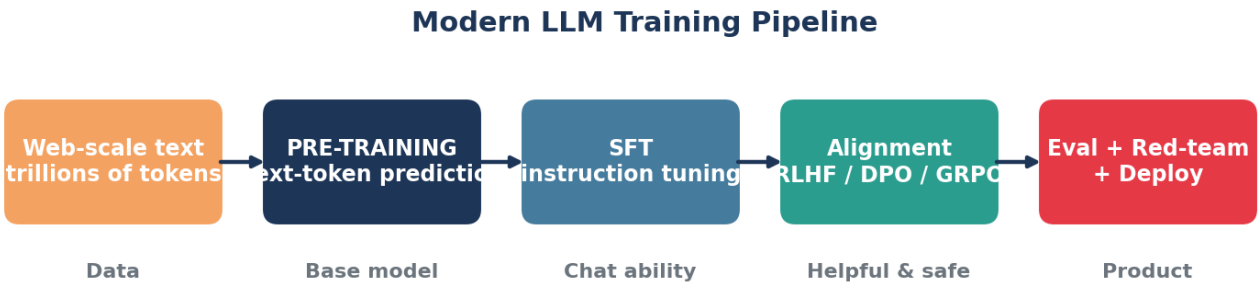


Figure 8.1 — End-to-end LLM development pipeline.

8.1 Pre-training & scaling laws

- **Data**: web crawl (deduplicated, quality-filtered), code, books, papers, multilingual, synthetic data. Deduplication, decontamination against benchmarks and PII removal matter as much as quantity.
- **Objective**: maximise $\Sigma \log p(x_t \mid x_{<t})$ over trillions of tokens. Emergent abilities (in-context learning, reasoning) appear with scale and data quality.
- **Scaling laws (Kaplan, Chinchilla)**: $L(N,D) = E + A/N^{\alpha} + B/D^{\beta}$. Compute-optimal ≈ **20 tokens per parameter**; but production models are deliberately *over-trained* (LLaMA-3-8B saw ~15T tokens) because inference cost dominates over the model's lifetime.

$$\text{Compute } C \approx 6\,N\,D \text{ FLOPs} \qquad \text{Optimal: } N \propto C^{0.5},\ D \propto C^{0.5}$$

8.2 Adaptation: prompting, fine-tuning, PEFT

Method	What changes	When to use	Cost
Prompting / few-shot / CoT	Nothing (context only)	First attempt always	≈ free
RAG	Adds retrieved knowledge to context	Fresh / private knowledge, citations	Low
Full fine-tuning (SFT)	All weights	Large data, deep behaviour change	Very high
LoRA	Low-rank adapters $\Delta W = BA$	Domain/style adaptation on modest hardware	Low
QLoRA	LoRA on 4-bit NF4 quantised base	Fine-tune 30–70B on a single GPU	Very low
Adapters / Prefix / Prompt tuning	Small inserted modules or soft prompts	Multi-task serving	Low
Distillation	Small student mimics large teacher	Latency / cost reduction	Medium

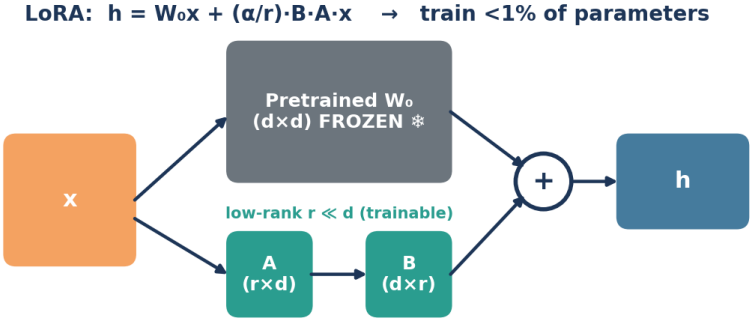


Figure 8.2 — LoRA freezes W_0 and learns a low-rank update; B is initialised to zero so training starts from the base model.

- LoRA hyper-parameters: rank r (8–64), α (often 2r), target modules (q,k,v,o and MLP), dropout. Adapters can be **merged** into W_0 for zero inference overhead or hot-swapped per tenant.

- **Rule of thumb**: fine-tune to teach *format, style, behaviour and skills*; use RAG to supply *facts*. Fine-tuning on new facts is unreliable and can increase hallucination.
- Common SFT pitfalls: low-quality or inconsistent data, format mismatch with chat template, catastrophic forgetting (mix in general data, lower LR), evaluating on training prompts.

8.3 Alignment: RLHF, DPO, GRPO

Method	Idea	Key equation / note
RLHF (PPO)	Train a reward model on human preference pairs; optimise policy with PPO + KL penalty to reference model	$\max E[r(x,y)] - \beta \cdot KL(\pi \| \pi_{ref})$. Needs 4 models in memory; unstable
DPO	Directly optimise preferences without RL or explicit reward model	$L = -\log \sigma(\beta[\log \pi(y_w\|x)/\pi_{ref}(y_w\|x) - \log \pi(y_l\|x)/\pi_{ref}(y_l\|x)])$
GRPO / RLVR	Sample a group of answers, advantage = (r_i − mean)/std within the group; verifiable rewards (unit tests, math answers); no value network	Powers recent reasoning models; reward hacking is the risk
Constitutional AI / RLAIF	AI feedback guided by written principles replaces some human labels	Scalable oversight
Best-of-N / rejection sampling	Sample N, keep highest-reward output for SFT or serving	Simple, strong baseline

ADVANCED **PPO clipped objective**: $L = E[\min(r_t(\theta)A_t , \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) A_t)]$, where $r_t = \pi_\theta(a|s)/\pi_{old}(a|s)$. The clip keeps each update inside a trust region without second-order machinery.

8.4 Decoding & sampling

Strategy	Mechanism	Effect
Greedy	argmax each step	Deterministic, repetitive
Beam search	Keep top-B partial sequences	Good for translation; bland for chat
Temperature T	softmax(z/T)	T<1 sharper, T>1 more random
Top-k	Sample from k most likely tokens	Cuts long tail
Top-p (nucleus)	Smallest set with cumulative prob ≥ p	Adaptive cut-off; p≈0.9
Min-p, typical, repetition/frequency penalties	Refined truncation and anti-repetition	Quality tuning
Constrained / grammar decoding	Mask tokens that break a JSON schema/regex	Guaranteed valid structured output

8.5 Efficient inference

- **Prefill vs decode**: prefill processes the prompt in parallel (compute-bound); decode generates one token at a time (**memory-bandwidth bound** — every step re-reads all weights + KV-cache).
- **KV-cache** stores past keys/values so each new token costs O(n) instead of recomputing $O(n^2)$. Managed with **PagedAttention** (vLLM) to avoid fragmentation; **continuous batching** inserts new requests every step; **prefix caching** reuses shared prompts.
- **Quantisation**: FP16/BF16 → FP8 / INT8 / INT4 (GPTQ, AWQ, GGUF) — 2–4× smaller and faster with small quality loss; KV-cache quantisation for long contexts.
- **Speculative decoding**: a small draft model proposes k tokens, the large model verifies them in one pass — same output distribution, 2–3× speed-up. Also Medusa/EAGLE heads.
- **Others**: FlashAttention, tensor-parallel serving, model distillation, sliding-window attention, MoE routing, request scheduling and autoscaling.

Metric	Meaning	Typical target
TTFT	Time to first token (prefill latency)	< 500 ms for chat
TPOT / ITL	Time per output token	20–50 ms (≈ 20–50 tok/s per user)

Metric	Meaning	Typical target
Throughput	Tokens/s across all requests	Maximise per GPU-$
Cost	$ per 1M input / output tokens	Output tokens cost 3–5× input

8.6 Evaluation, hallucination, safety

- **Benchmarks**: MMLU/MMLU-Pro (knowledge), GSM8K/MATH (reasoning), HumanEval/SWE-bench (code), IFEval (instruction following), MT-Bench / Arena Elo (chat preference), long-context needle tests. Beware **contamination** and saturation; always build a **task-specific eval set**.
- **LLM-as-judge**: cheap and scalable but biased (position, verbosity, self-preference). Use pairwise comparisons, swap order, calibrate against human labels.
- **Hallucination** mitigation: retrieval grounding + citations, lower temperature, ask for uncertainty, self-consistency / verification steps, tool use, constrained output, abstain option, fine-tune on refusals.
- **Safety**: prompt injection (direct & indirect), jailbreaks, data exfiltration, PII leakage. Defences: input/output classifiers, privilege separation, tool allow-lists, sandboxing, human approval for risky actions, red-teaming.

8.7 Prompt engineering patterns

Pattern	Description
Few-shot	Provide 2–8 input→output examples matching format and edge cases
Chain-of-Thought	Ask the model to reason step-by-step before answering; boosts maths & logic
Self-consistency	Sample several reasoning paths, majority-vote the answer
ReAct	Interleave Thought → Action (tool call) → Observation
Role / system prompt	Fix persona, rules, format; put stable instructions first (enables prefix caching)
Structured output	JSON schema / function-calling; validate and retry on parse failure
Decomposition / prompt chaining	Split a complex task into verified stages

9. RAG, Vector Search & AI Agents

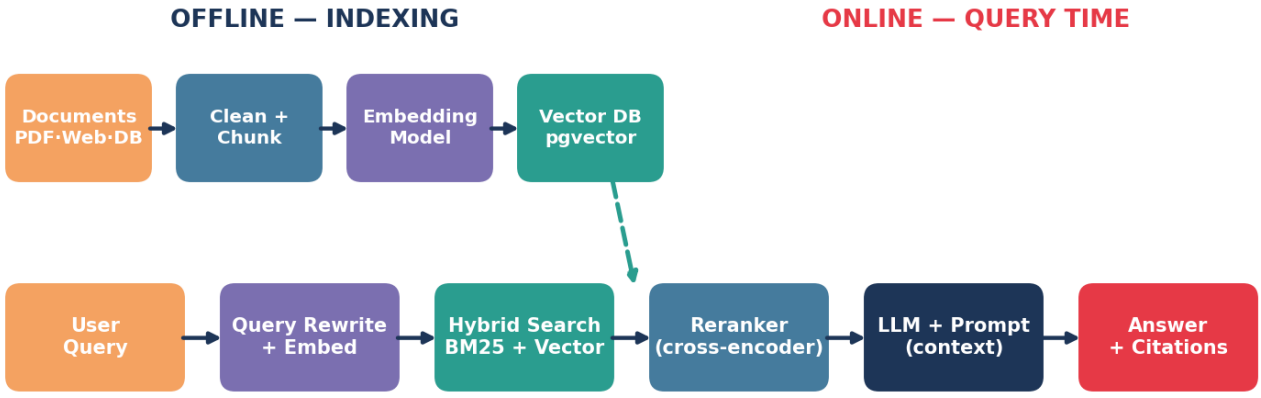


Figure 9.1 — Production RAG: offline indexing and online retrieval-augmented generation.

9.1 Building blocks of a strong RAG system

Stage	Decisions	Best practice
Ingestion	Parsing PDFs/HTML/tables, OCR, metadata	Preserve structure & source; keep doc id, page, section, ACLs
Chunking	Size, overlap, boundaries	200–800 tokens, 10–20% overlap; split on headings/sentences; parent-document retrieval
Embeddings	General vs domain model, dimension	BGE / E5 / GTE / OpenAI-class; normalise; instruction prefixes; fine-tune on domain pairs if needed
Index	Flat, IVF, HNSW, PQ	HNSW for low-latency recall; IVF-PQ for billion-scale memory savings
Retrieval	Dense, sparse (BM25), hybrid	**Hybrid** = dense + BM25 fused with RRF; add metadata filters
Reranking	Cross-encoder scores query+doc jointly	Retrieve top-50 → rerank → keep top-5
Generation	Prompt template, citations, refusal	Instruct to answer only from context, cite ids, say 'not found'
Evaluation	Retrieval + generation	Recall@k, MRR, nDCG; faithfulness, answer relevance, context precision (RAGAS-style)

Reciprocal Rank Fusion: $\text{RRF}(d) = \sum_{r \in \text{rankers}} 1 / (k + \text{rank}_r(d))$, $k \approx 60$

ANN index	Idea	Trade-off
Flat (brute force)	Exact scan	100% recall, O(N) — fine up to ~10^5–10^6 vectors
IVF	k-means partitions; search *nprobe* nearest cells	Fast; recall depends on nprobe
PQ / OPQ	Compress vectors to short codes	10–50× less memory, lossy
HNSW	Layered small-world graph; params M, efConstruction, efSearch	Best recall/latency; high RAM
DiskANN / ScaNN	SSD-based / anisotropic quantisation	Billion-scale, cost-efficient

- **Bi-encoder** (embed separately, fast, used for retrieval) vs **cross-encoder** (joint attention, accurate, slow, used for reranking).
- **Advanced RAG**: query rewriting / multi-query, HyDE (embed a hypothetical answer), query decomposition for multi-hop, contextual chunk headers, GraphRAG (entity graph), self-RAG / corrective RAG, agentic RAG that decides when and what to retrieve.

- **Failure analysis**: separate *retrieval failures* (right chunk not in top-k) from *generation failures* (chunk present, answer wrong). Watch for 'lost in the middle' — put the most relevant chunks first/last.
- **Production**: cache embeddings and answers, stream tokens, per-tenant access control, incremental re-indexing, freshness policies, observability (trace every retrieval), cost/latency budgets.

INTERVIEW TIP When asked 'RAG vs fine-tuning?' answer: RAG for **knowledge** (updatable, attributable, secure), fine-tuning for **behaviour/format**, and often **both**. Then add how you would measure each.

9.2 AI agents

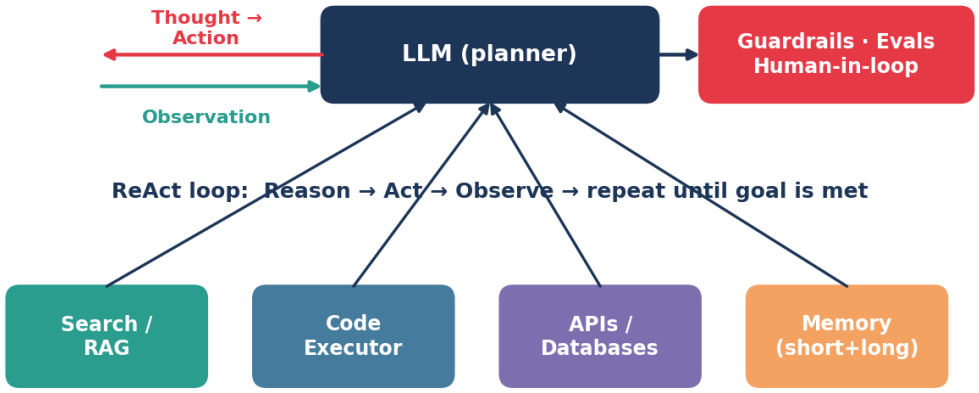


Figure 9.2 — An LLM agent plans, calls tools, observes results and iterates within guardrails.

- **Agent** = LLM + tools + memory + control loop. Tools are described by JSON schemas (function calling); open standards like **Model Context Protocol (MCP)** standardise tool/data connections.
- **Patterns**: ReAct; plan-and-execute; reflection / self-critique; router (pick a specialist); orchestrator-workers; multi-agent debate; human-in-the-loop approval for irreversible actions.
- **Memory**: short-term (context window), long-term (vector store / database), episodic summaries; manage context growth with summarisation and retrieval.
- **Reliability**: constrain tool permissions, validate arguments, idempotent tools, timeouts and retries, step limits to prevent loops, deterministic workflows wherever a fixed pipeline suffices.
- **Evaluation**: task success rate, trajectory/tool-call correctness, cost, latency, safety incidents; build regression suites of scripted scenarios and trace every run.

KEY IDEA Prefer the **simplest architecture that works**: single prompt → prompt chain → workflow with tools → autonomous agent. Every extra degree of autonomy multiplies failure modes and cost.

10. Generative Models

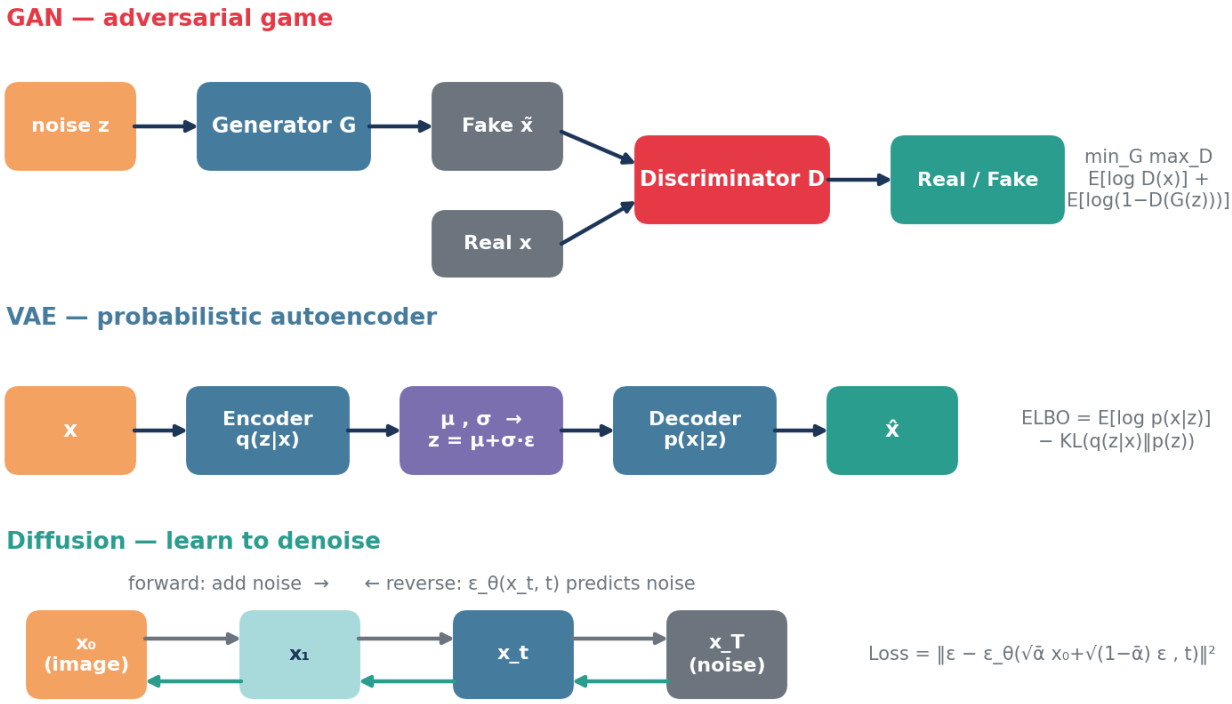


Figure 10.1 — GAN, VAE and diffusion at a glance.

Model	Training	Sampling	Strengths	Weaknesses
GAN	Min-max game between G and D	1 forward pass (fast)	Sharp images, fast inference	Mode collapse, unstable; WGAN-GP, spectral norm help
VAE	Maximise ELBO with reparameterisation $z=\mu+\sigma\odot\varepsilon$	1 pass	Stable, smooth latent space	Blurry samples
Autoregressive	Next-token / next-patch likelihood	Sequential	Exact likelihood; scales (LLMs)	Slow sampling
Normalising flows	Invertible maps, exact likelihood	1 pass	Exact density	Architecture constraints
Diffusion	Predict noise ε (denoising score matching)	20–50 steps (DDIM, DPM++)	State-of-the-art quality, diverse, controllable	Slower; distillation / consistency models speed up

10.1 Diffusion in detail

$$\text{Forward: } x_t = \sqrt{\bar\alpha_t}\cdot x_0 + \sqrt{(1-\bar\alpha_t)}\cdot\varepsilon\ ,\ \varepsilon\sim N(0,I) \qquad \text{Loss: } E\,\|\varepsilon - \varepsilon_\theta(x_t, t, c)\|^2$$

- **Latent diffusion** (Stable Diffusion): a VAE compresses images (×8 per side) and diffusion runs in latent space — ~50× cheaper. Denoiser is a U-Net or **Diffusion Transformer (DiT)**; text conditioning through cross-attention with CLIP/T5 embeddings.
- **Classifier-free guidance**: $\tilde\varepsilon = \varepsilon_{uncond} + w\cdot(\varepsilon_{cond} - \varepsilon_{uncond})$; larger w → stronger prompt adherence, lower diversity.
- **Flow matching / rectified flow** learn straight-line velocity fields → fewer steps; used by recent image/video models.
- **Control & personalisation**: ControlNet (spatial conditions), LoRA / DreamBooth (subject/style), inpainting, image-to-image.

COMMON TRAP GAN discriminator loss going to zero is a **bad** sign — G is not learning. Monitor sample quality (FID), not just the losses.

11. Reinforcement Learning

Goal: maximise expected discounted return $J(\pi) = E[\ \Sigma\ \gamma^t\ r_t\ ]$



Bellman: $V(s) = \max_a [\ R(s,a) + \gamma\ \Sigma\ P(s'|s,a)\ V(s')\ ]$

Figure 11.1 — The agent–environment loop of a Markov Decision Process.

- **MDP** = (S, A, P, R, γ). **Policy** π(a|s); **value** $V^{\pi}(s)=E[\Sigma\gamma^{t}r_{t}]$; **action-value** $Q^{\pi}(s,a)$; **advantage** A=Q−V.
- **Exploration vs exploitation**: ε-greedy, UCB, Thompson sampling, entropy bonuses. **Multi-armed bandit** = RL without state — used in ads, recommendations and A/B allocation.

Family	Algorithms	Key points
Value-based	Q-learning: $Q(s,a) \leftarrow Q + \alpha[r + \gamma\ \max_{a'} Q(s',a') - Q]$; SARSA; **DQN** (replay buffer + target network)	Off-policy; discrete actions; overestimation → Double DQN
Policy gradient	REINFORCE: $\nabla J = E[\nabla \log \pi(a\|s) \cdot G_t]$	High variance → subtract baseline
Actor-critic	A2C/A3C, **PPO**, TRPO, DDPG, TD3, **SAC**	Critic estimates V/Q to reduce variance; PPO clip = stable default; SAC for continuous control
Model-based	Dyna, MuZero, Dreamer	Learn dynamics to plan → sample-efficient
Offline / imitation	Behaviour cloning, CQL, DAgger, inverse RL	Learn from logged data without exploration

Bellman optimality: $Q^*(s,a) = E[\ r + \gamma \cdot \max_{a'} Q^*(s',a')\ ]$

- **On-policy** (PPO) needs fresh data but is stable; **off-policy** (DQN, SAC) reuses replay data and is sample-efficient.
- **Credit assignment**, sparse rewards, reward hacking and sim-to-real gaps are the practical challenges; use reward shaping carefully.
- **Link to LLMs**: RLHF treats generation as a bandit/MDP where the reward model scores the completion; GRPO/RLVR use verifiable rewards to train reasoning.

12. MLOps & ML System Design

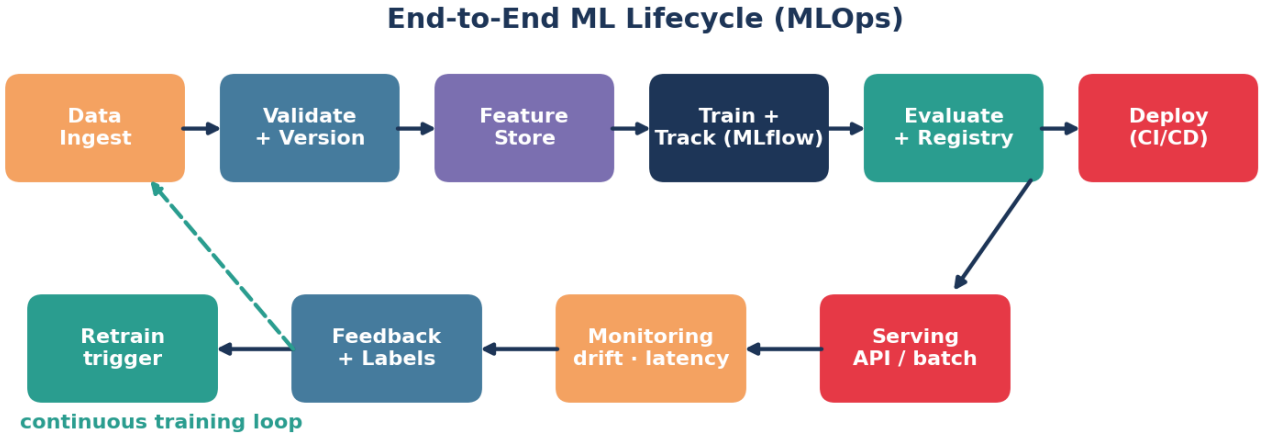


Figure 12.1 — End-to-end ML lifecycle with a continuous-training feedback loop.

12.1 MLOps building blocks

Layer	Purpose	Tools (examples)
Data versioning & validation	Reproducible datasets; schema/quality checks	DVC, lakeFS, Great Expectations, TFDV
Feature store	Same features offline (training) and online (serving) → no train/serve skew	Feast, Tecton, Databricks FS
Experiment tracking & registry	Params, metrics, artifacts, lineage, staged promotion	MLflow, Weights & Biases, Vertex/SageMaker
Pipelines / orchestration	Automated, retryable DAGs	Airflow, Kubeflow, Dagster, Prefect
Serving	REST/gRPC, batch, streaming, LLM servers	FastAPI, TorchServe, Triton, vLLM, BentoML, KServe
Monitoring	Latency, errors, data & concept drift, quality, cost	Prometheus, Grafana, Evidently, WhyLabs, Arize, LangSmith
Infra	Containers, GPUs, autoscaling, IaC	Docker, Kubernetes, Terraform, Ray

12.2 Serving patterns & deployment strategies

Pattern	When	Notes
Batch prediction	Latency-insensitive (nightly scores, email campaigns)	Cheap, simple; results stored in DB/cache
Online (real-time)	Per-request < 100 ms (fraud, ranking)	Feature store lookup, autoscale, cache, fall-backs
Streaming	Event-driven (Kafka/Flink features)	Windowed aggregates; exactly-once semantics
Edge / on-device	Privacy, offline, ultra-low latency	Quantise, distil, TFLite / Core ML / ONNX

Rollout strategy	Description	Risk control
Shadow	New model receives traffic, outputs are logged, not served	Zero user risk; compare offline
Canary	1% → 5% → 25% → 100% of traffic	Auto-rollback on metric regression
Blue-green	Two full environments, switch traffic	Instant rollback
A/B test	Randomised split, measure business KPI	Statistical rigour; guardrail metrics
Interleaving / bandits	Mix rankings or adaptively allocate	Faster for ranking; less traffic

12.3 Monitoring & drift

- **Data drift** (P(X) changes), **concept drift** (P(y|X) changes), **label / prior shift** (P(y) changes), **upstream data bugs** (schema, units, nulls).
- Detect with **PSI** = $\Sigma(a_i - e_i)\cdot\ln(a_i/e_i)$ (<0.1 stable, 0.1–0.25 moderate, >0.25 major), KS test, JS-divergence, embedding drift; track prediction distribution and delayed-label performance.
- Retraining triggers: schedule, drift threshold, performance drop, new data volume. Always keep a rollback model and a champion/challenger comparison.
- **Testing ML**: data tests, unit tests for feature code, invariance and directional tests, slice-based evaluation (per segment fairness), load tests, and shadow comparisons.

12.4 ML system-design framework (use in every design round)

Step	Questions to answer
1. Clarify	Business goal, users, scale (QPS, data size), latency, cost, privacy, constraints
2. Metrics	Business KPI ↔ online metric ↔ offline metric ↔ guardrails (latency, fairness, safety)
3. Data	Sources, labels (implicit/explicit), volume, freshness, bias, privacy, labelling cost
4. Features & model	Baseline (rules/GBDT) → advanced model; embeddings; multi-stage pipeline
5. Training	Splits (time-based), sampling/negatives, loss, distributed training, retrain cadence
6. Serving	Online vs batch, caching, feature store, ANN, autoscaling, fallbacks
7. Evaluation	Offline metrics, A/B design, sample size, novelty/ramp-up effects
8. Monitoring & iteration	Drift, feedback loops, failure modes, cold-start, abuse, roadmap

12.5 Worked design sketches

Problem	Key design choices	Metrics	Pitfalls
Fraud detection (payments, 20 ms budget)	Streaming features (velocity, device, graph); GBDT + rules; threshold by cost matrix; human review queue; delayed labels (chargebacks)	PR-AUC, recall@fixed FPR, $ saved, review load	Extreme imbalance, label delay, adversarial adaptation, concept drift
Feed / video recommender	Two-tower retrieval (ANN) → ranker (multi-task DNN: click, watch-time, like) → re-rank (diversity, safety)	CTR, watch-time, retention (online); recall@k, NDCG (offline)	Position & selection bias, feedback loops, cold-start, popularity bias
Enterprise RAG assistant	Ingestion + ACL metadata; hybrid retrieval + reranker; cited answers; guardrails; caching; feedback capture	Recall@k, faithfulness, answer accuracy, deflection, latency, cost/query	Stale index, permission leaks, prompt injection, hallucinated citations
Search ranking	BM25 + embeddings recall; LambdaMART / cross-encoder rank; query understanding	NDCG@10, MRR, CTR, abandonment	Click bias, query drift, long-tail queries
LLM serving platform	Continuous batching (vLLM), quantised weights, prefix cache, autoscaling on queue depth, multi-model routing	TTFT, TPOT, tokens/s/$, error rate	GPU under-utilisation, long-context spikes, noisy neighbours

13. Distributed Training & Performance Engineering

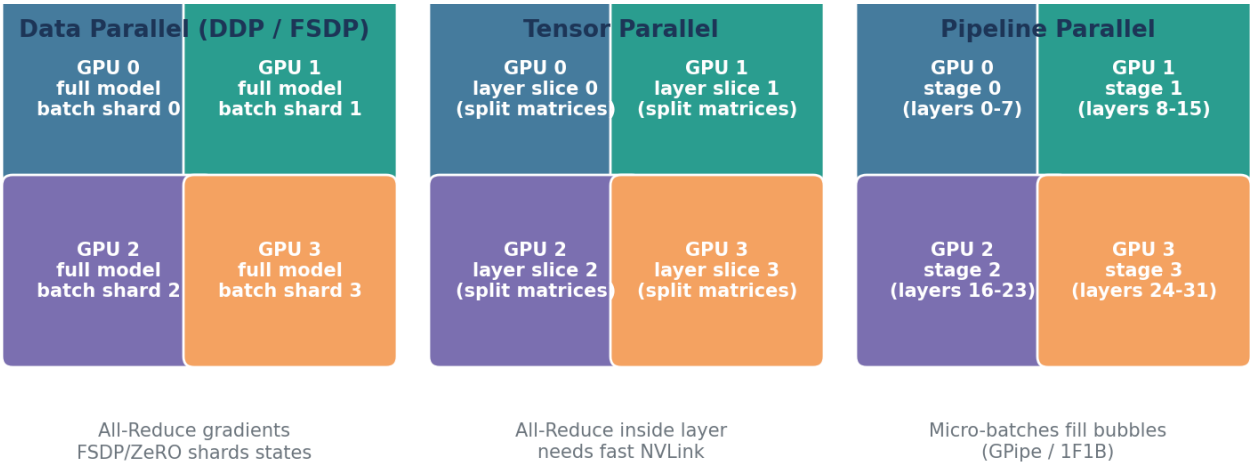


Figure 13.1 — Data, tensor and pipeline parallelism (combined as 3D parallelism for frontier models).

Strategy	What is split	Communication	Use when
Data parallel (DDP)	Batch; full model copy per GPU	All-reduce gradients each step	Model fits on one GPU
ZeRO-1/2 / FSDP	Optimiser states → gradients → parameters sharded	All-gather params, reduce-scatter grads	Model too big for one GPU but layers fit
Tensor parallel	Individual matrices (column/row split)	All-reduce per layer (needs NVLink)	Very large layers; within a node
Pipeline parallel	Consecutive layers per stage	Point-to-point activations; micro-batches hide bubbles	Across nodes; deep models
Expert parallel	MoE experts on different GPUs	All-to-all token routing	MoE models
Sequence / context parallel	Sequence dimension (ring attention)	KV blocks passed around ring	Million-token contexts

13.1 Memory budgeting

> Mixed-precision Adam: 2 (bf16 weights) + 2 (grads) + 4 (fp32 master) + 4 (m) + 4 (v) = **16 bytes / parameter**

- A 7B model needs ≈ 7×16 = **112 GB** for states alone (before activations) → shard with ZeRO-3/FSDP or use LoRA/QLoRA. Inference in fp16 ≈ 2 bytes/param = 14 GB; INT4 ≈ 3.5 GB.
- **Activation memory** scales with batch × sequence × layers; reduce via **gradient checkpointing** (recompute, ≈ +30% compute), FlashAttention, smaller micro-batches + gradient accumulation.
- **Mixed precision**: bf16 (same range as fp32, no loss-scaling) is preferred; fp16 needs dynamic loss scaling; FP8 on H100-class GPUs for higher throughput.

13.2 Performance checklist

- **Roofline thinking**: compute-bound (large matmuls) vs memory-bound (element-wise ops, decoding). Fuse kernels (torch.compile, Triton), use channels-last, pinned memory and async data loading.
- **Utilisation metric**: MFU = achieved FLOPs / peak FLOPs; 40–55% is excellent for LLM training.
- **Data pipeline** is often the bottleneck: parallel workers, prefetch, sharded formats (WebDataset, Parquet, Arrow), caching, avoid tiny files.
- **Debugging scale**: check loss spikes, straggler GPUs, NCCL timeouts, non-determinism, checkpoint/resume correctness, and learning-rate/batch-size scaling.
- **Compression for deployment**: quantisation (PTQ vs QAT), pruning (structured), distillation, low-rank factorisation, export via ONNX/TensorRT, dynamic batching.

INTERVIEW TIP Interviewers love back-of-envelope estimates: 'How many GPUs to train a 70B model on 2T tokens in 30 days?' → FLOPs = 6·70e9·2e12 = 8.4e23; at ~400 TFLOP/s peak × 40% MFU ≈ 1.6e14 FLOP/s per GPU → 5.2e9 GPU-s ≈ 1.46M GPU-hours → over 720 h ≈ **~2,000 GPUs**.

14. Coding Toolkit for ML Interviews

14.1 Multi-head causal self-attention (PyTorch)

```
import math, torch, torch.nn as nn, torch.nn.functional as F

class CausalSelfAttention(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.0):
        super().__init__()
        assert d_model % n_heads == 0
        self.h, self.dk = n_heads, d_model // n_heads
        self.qkv = nn.Linear(d_model, 3 * d_model, bias=False)
        self.out = nn.Linear(d_model, d_model, bias=False)
        self.drop = nn.Dropout(dropout)

    def forward(self, x):                       # x: (B, T, C)
        B, T, C = x.shape
        q, k, v = self.qkv(x).chunk(3, dim=-1)
        q, k, v = [t.view(B, T, self.h, self.dk).transpose(1, 2) for t in (q, k, v)]  # (B,h,T,dk)
        att = (q @ k.transpose(-2, -1)) / math.sqrt(self.dk)                          # (B,h,T,T)
        mask = torch.triu(torch.ones(T, T, device=x.device, dtype=torch.bool), 1)
        att = att.masked_fill(mask, float("-inf")).softmax(dim=-1)
        y = self.drop(att) @ v                                                        # (B,h,T,dk)
        return self.out(y.transpose(1, 2).contiguous().view(B, T, C))
# Production: use F.scaled_dot_product_attention(q, k, v, is_causal=True)  -> FlashAttention kernel
```

14.2 Standard PyTorch training loop (AMP, clipping, scheduler)

```
model.train()                               # bf16 autocast -> no GradScaler required
for step, (x, y) in enumerate(loader):
    x, y = x.to(dev, non_blocking=True), y.to(dev, non_blocking=True)
    with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
        loss = criterion(model(x), y) / accum_steps
    loss.backward()
    if (step + 1) % accum_steps == 0:
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step(); scheduler.step(); optimizer.zero_grad(set_to_none=True)
    if step % 100 == 0: print(step, loss.item() * accum_steps)
@torch.no_grad()
def evaluate(model, loader):
    model.eval(); tot = n = 0
    for x, y in loader:
        p = model(x.to(dev)).argmax(-1).cpu(); tot += (p == y).sum().item(); n += len(y)
    return tot / n
```

14.3 Logistic regression & k-means from scratch (NumPy)

```
import numpy as np
def sigmoid(z): return 1 / (1 + np.exp(-np.clip(z, -50, 50)))

def fit_logreg(X, y, lr=0.1, epochs=500, l2=1e-3):
    n, d = X.shape; w, b = np.zeros(d), 0.0
    for _ in range(epochs):
        p = sigmoid(X @ w + b)
        w -= lr * (X.T @ (p - y) / n + l2 * w)      # gradient of BCE + L2
        b -= lr * np.mean(p - y)
    return w, b

def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    C = X[rng.choice(len(X), k, replace=False)]
    for _ in range(iters):
        lab = np.argmin(((X[:, None, :] - C[None]) ** 2).sum(-1), axis=1)   # assign
        newC = np.array([X[lab == j].mean(0) if np.any(lab == j) else C[j] for j in range(k)])
        if np.allclose(newC, C): break
        C = newC
    return C, lab
```

14.4 LoRA layer & ranking metrics

```
class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, r=8, alpha=16):
        super().__init__()
        self.base = base
        for p in self.base.parameters(): p.requires_grad = False       # freeze W0
        self.A = nn.Parameter(torch.randn(r, base.in_features) * 0.01)
        self.B = nn.Parameter(torch.zeros(base.out_features, r))       # zero-init -> starts as base
        self.scale = alpha / r
    def forward(self, x):
        return self.base(x) + (x @ self.A.T @ self.B.T) * self.scale

def ndcg_at_k(rels, k):                    # rels: graded relevance in ranked order
    rels = np.asarray(rels, float)[:k]
    dcg = ((2 ** rels - 1) / np.log2(np.arange(2, len(rels) + 2))).sum()
    ideal = np.sort(rels)[::-1]
    idcg = ((2 ** ideal - 1) / np.log2(np.arange(2, len(ideal) + 2))).sum()
    return dcg / idcg if idcg > 0 else 0.0
def rrf(rankings, k=60):                   # rankings: list of ranked doc-id lists
    score = {}
    for r in rankings:
        for pos, d in enumerate(r, 1): score[d] = score.get(d, 0) + 1 / (k + pos)
    return sorted(score, key=score.get, reverse=True)
```

14.5 SQL & pandas patterns frequently asked

```
-- Top-3 products per category by revenue (window function)
SELECT * FROM (
  SELECT category, product, SUM(revenue) AS rev,
         DENSE_RANK() OVER (PARTITION BY category ORDER BY SUM(revenue) DESC) AS rk
  FROM sales GROUP BY category, product) t
WHERE rk <= 3;

# pandas: 7-day rolling mean per user, then lag feature, leak-free by shift
df = df.sort_values(["user", "ts"])
df["roll7"] = df.groupby("user")["amount"].transform(lambda s: s.shift(1).rolling(7, min_periods=1).mean())
df["prev_amt"] = df.groupby("user")["amount"].shift(1)
```

14.6 Algorithm & complexity cheat-sheet

Pattern	Typical problems	Complexity note
Two pointers / sliding window	Subarray sums, longest substring	O(n)
Hash map / counting	Two-sum, top-k frequent, dedupe	O(n) time, O(n) space
Heap / priority queue	Top-k, streaming median, merge k lists	O(n log k)
Binary search (on answer)	Search rotated array, capacity problems	O(log n)
BFS / DFS / topological sort	Graphs, grids, dependency order	O(V+E)
Dynamic programming	Knapsack, edit distance, LIS	State × transition cost
Sorting / intervals	Merge intervals, meeting rooms	O(n log n)
Matrix ops	Cosine similarity, batch distance, softmax	Vectorise; avoid Python loops

INTERVIEW TIP In ML coding rounds: (1) restate the problem & shapes, (2) write a naive correct version, (3) vectorise, (4) test on a tiny example, (5) discuss numerical stability (log-sum-exp, clipping) and complexity.

15. Interview Q&A Bank

15.1 Machine-learning fundamentals

Q. Explain bias vs variance and how you would diagnose each.
Bias = error from wrong/simple assumptions (underfit); variance = sensitivity to training sample (overfit). Compare train vs validation error and use learning curves: high both → bias; big gap → variance. Fix bias with capacity/features; fix variance with data, regularisation, ensembling.

Q. Why does L1 give sparsity but L2 does not?
The L1 constraint region has corners on the axes, so the optimum often lands where coordinates are exactly zero; the L2 ball is smooth, so it only shrinks. Equivalently the L1 subgradient is a constant push toward zero while L2's shrinks proportionally to the weight.

Q. Precision vs recall — how do you choose a threshold?
Plot the PR curve, attach costs to FP and FN, and choose the operating point minimising expected cost or satisfying a constraint (e.g. recall ≥ 95%). Never default to 0.5, and recalibrate if probabilities are used downstream.

Q. Random forest vs gradient boosting?
RF: parallel deep trees, averaging reduces variance, robust with little tuning. GBM: shallow trees fitted sequentially to gradients, reduces bias, usually higher accuracy but needs tuning (η, depth, subsampling) and can overfit noisy labels.

Q. How do you handle missing data and categorical variables with high cardinality?
Impute with indicators (or use native NaN handling in trees); for high-cardinality categoricals use target encoding with out-of-fold statistics and smoothing, hashing, or learned embeddings; guard against leakage.

Q. What is data leakage? Give three examples.
Using information unavailable at prediction time: features derived from the target, fitting normalisation on the full dataset, and random splitting of time-series/grouped data. Detect through suspiciously high metrics and feature-importance audits.

Q. Explain PCA and when it fails.
PCA projects onto orthogonal directions of maximal variance (top eigenvectors of covariance / SVD). It is linear, scale-sensitive and variance ≠ relevance — a low-variance direction may be the discriminative one; use supervised methods (LDA) or non-linear embeddings.

Q. Why is accuracy misleading on imbalanced data?
A trivial majority-class predictor scores 99% when positives are 1%. Use PR-AUC, recall/precision at a threshold, F-scores, MCC and cost-based metrics.

Q. How does k-fold cross-validation work and when not to use it?
Split into k folds, train on k−1, validate on the remaining, average. Avoid for time series (use forward chaining), grouped data (use group k-fold) or very expensive models (use a hold-out).

15.2 Deep learning

Q. Why is ReLU preferred over sigmoid?
Non-saturating for positive inputs (gradient 1), cheap, sparse activations; sigmoid saturates causing vanishing gradients and is not zero-centred.

Q. Explain BatchNorm — and why LayerNorm in Transformers?
BN normalises per channel across the batch, smoothing the loss landscape, allowing higher LR. It depends on batch statistics (bad for small batches/variable lengths, train≠eval). LN normalises each token across features — batch-independent and suited to sequences.

Q. What does Adam do and why AdamW?
Adam keeps exponential moving averages of the gradient (momentum) and squared gradient (scale) giving per-parameter adaptive steps. L2 penalty inside Adam is rescaled by the adaptive term; AdamW decouples weight decay from the gradient update, restoring true regularisation.

Q. How do you prevent/handle exploding gradients?
Gradient-norm clipping, lower LR with warm-up, correct initialisation, normalisation layers, residual connections, careful mixed-precision (bf16).

Q. Why do residual connections help?
They provide an identity path so gradients flow to early layers ($\partial y/\partial x = \partial F/\partial x + I$) and let layers learn residual corrections, easing optimisation of very deep nets.

Q. Your model's training loss is NaN — what do you check?
Data (NaN/inf, extreme values), learning rate too high, log(0) or division by zero, fp16 overflow, bad init, exploding gradients. Reproduce on a small batch, enable anomaly detection, log grad norms.

Q. Explain the vanishing-gradient problem in RNNs and the LSTM fix.
Gradients are products of Jacobians over time and shrink exponentially. LSTM adds an additive cell-state path controlled by gates so the derivative along the cell state can stay close to 1.

Q. Describe dropout at train vs inference.
Train: zero each unit with prob p and scale by 1/(1−p). Inference: use all units unchanged (inverted dropout). Acts like averaging many sub-networks.

15.3 Transformers & LLMs

Q. Walk through self-attention and its complexity.
Project X to Q,K,V; scores = $QK^T/\sqrt{d}$; softmax with mask; multiply by V. $O(n^2d)$ time, $O(n^2)$ memory. FlashAttention removes the n^2 memory via tiling; sparse/linear attention and SSMs reduce compute.

Q. What is the KV-cache and why does decoding become memory-bound?
Stores keys/values of past tokens so each step only computes attention for the new token. Each decode step reads all weights and the cache but performs few FLOPs — arithmetic intensity is low, so bandwidth limits speed. Mitigate with GQA, quantisation, batching, paged caches.

Q. Explain RoPE.
Rotary embeddings rotate each pair of query/key dimensions by an angle proportional to position; the dot-product then depends only on relative distance. Scaling the frequencies (PI, NTK, YaRN) extends context length.

Q. LoRA — why does low rank suffice?
Fine-tuning updates have low intrinsic dimension: $\Delta W \approx BA$ with $r \ll d$ captures most adaptation. It trains <1% of parameters, needs little optimiser memory and can be merged for inference.

Q. RLHF vs DPO?
RLHF fits a reward model then optimises the policy with PPO under a KL constraint — powerful but complex. DPO derives a closed-form classification-style loss on preference pairs relative to a reference model — simpler, stable, but offline and dependent on pair quality.

Q. How do you reduce hallucinations in a production LLM app?
Ground with retrieval, require citations, restrict to provided context, lower temperature, add verification (second pass or tool-based checks), allow abstention, evaluate with a labelled set, and monitor user feedback.

Q. How would you evaluate a RAG system?
Retrieval: recall@k, MRR, nDCG on labelled query→chunk pairs. Generation: faithfulness (claims supported by context), answer relevance, correctness vs gold answers, citation accuracy. Plus latency, cost and human review of failures.

Q. Long-context vs RAG?
Long context is simple but costly (n^2 attention, KV memory) and suffers 'lost in the middle'; RAG is cheaper and updatable but adds retrieval error. Use RAG for large/changing corpora, long context for deep reasoning over a few documents — often combine.

Q. What is speculative decoding and is it lossless?
A cheap draft model proposes several tokens; the target model verifies them in parallel and accepts a prefix using a rejection-sampling rule that preserves the target distribution exactly — lossless, gives ≈2–3× speed-up.

15.4 MLOps & system design

Q. How do you detect and respond to model drift?
Monitor input feature distributions (PSI/KS), prediction distribution and delayed-label metrics; alert on thresholds; investigate upstream changes; retrain or roll back via champion/challenger and canary release.

Q. What is training-serving skew and how to prevent it?
Differences between features at training and serving (code paths, time windows, missing values). Prevent with a shared feature store/transform code, logging serving features for training, and integration tests comparing both paths.

Q. Design an online-learning or continuous-training system.
Log predictions + features + outcomes, join delayed labels, validate data, retrain on schedule/trigger, evaluate against champion on fresh slices, gated automatic promotion, staged rollout, monitoring and rollback.

Q. How do you A/B test a ranking model?

Randomise by user, define primary KPI + guardrails, compute sample size from MDE, run full weekly cycles, check SRM (sample-ratio mismatch), use CUPED for variance reduction, beware novelty/network effects.

Q. Cost-optimise an LLM feature.

Route easy queries to small models, cache (semantic + prefix), shorten prompts, batch, quantise/distil, cap output tokens, use RAG instead of long contexts, and monitor $/successful task.

15.5 Behavioural questions (STAR)

Question	What to demonstrate
Tell me about a project you're proud of	Problem → your specific contribution → measurable impact (latency −40%, F1 +8 pts)
A time a model failed in production	Detection, root cause, fix, prevention (monitoring, tests)
Disagreement with a teammate/stakeholder	Data-driven persuasion, empathy, outcome
Ambiguous requirements	Clarifying questions, MVP, iteration, success metric
Trade-off between accuracy and latency/cost	Quantify both sides, choose with business context
How do you stay current?	Papers (arXiv), reproduction, side projects, open-source contributions

16. Cheat Sheets & A–Z Glossary

16.1 Formula sheet

Concept	Formula
MSE / RMSE	$(1/n)\Sigma(y-\hat{y})^2$; $\sqrt{\text{MSE}}$
Binary cross-entropy	$-[y \log p + (1-y) \log(1-p)]$
Softmax	$e^{z_i}/\Sigma_j e^{z_j}$
Sigmoid derivative	$\sigma'(z) = \sigma(z)(1-\sigma(z))$
Linear regression (OLS)	$w = (X^TX)^{-1}X^Ty$
Ridge	$w = (X^TX + \lambda I)^{-1}X^Ty$
Gini / Entropy	$1-\Sigma p_i^2$; $-\Sigma p_i \log p_i$
Bayes rule	$P(A\|B) = P(B\|A)P(A)/P(B)$
F1	$2 \cdot P \cdot R/(P+R)$
Attention	$\text{softmax}(QK^T/\sqrt{d_k})V$
Adam step	$\theta \leftarrow \theta - \eta\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$
Conv output size	$\lfloor (N+2P-K)/S \rfloor + 1$
KL divergence	$\Sigma\ p \log(p/q)$
Perplexity	$\exp(-(1/T)\Sigma \log p(x_t\|x_{<t}))$
ELBO	$E_q[\log p(x\|z)] - KL(q(z\|x)\|p(z))$
DPO loss	$-\log \sigma(\beta[\Delta\log \pi(y_w) - \Delta\log \pi(y_l)])$
LoRA	$W = W_0 + (\alpha/r)BA$
Training FLOPs	$\approx 6\ N\ D$
Bellman	$V(s) = \max_a[R + \gamma\Sigma P\ V(s')]$

16.2 Default hyper-parameters (good starting points)

Model	Starting values
Gradient boosting	η 0.05–0.1, depth 4–8, subsample 0.8, colsample 0.8, early stopping 50 rounds
Random forest	n_estimators 300–500, max_features √d, min_samples_leaf 1–5
CNN fine-tuning	AdamW/SGD-momentum, LR 1e-4 (head 1e-3), cosine schedule, augmentations, batch 32–256
Transformer training	AdamW β=(0.9,0.95), wd 0.1, LR 3e-4→1e-5 cosine, warm-up 1–2%, grad-clip 1.0, bf16
LoRA fine-tune	r 16, α 32, dropout 0.05, LR 1e-4–2e-4, 1–3 epochs, all linear layers
RAG	chunk 400 tokens / 15% overlap, top-k 30 → rerank → 5, hybrid α 0.5, temperature 0–0.2

16.3 A–Z Glossary

Term	Meaning	Term	Meaning
Activation	Non-linear function applied to a layer's output	**MoE**	Mixture of Experts, sparse conditional computation
Attention	Mechanism weighting tokens by relevance via Q·K	**MLE**	Maximum likelihood estimation
Bagging	Bootstrap aggregating to reduce variance	**Normalisation**	Rescaling activations/features for stable training
Batch size	Samples per gradient step	**Overfitting**	Fits noise; poor generalisation
Bias (model)	Systematic error from restrictive assumptions	**PEFT**	Parameter-efficient fine-tuning
BPE	Byte-pair-encoding tokeniser	**Perplexity**	exp(cross-entropy); LM quality
Calibration	Agreement between predicted and empirical probabilities	**Quantisation**	Lower-precision weights/activations
Chain-of-Thought	Prompting for intermediate reasoning steps	**RAG**	Retrieval-augmented generation
Curse of dimensionality	Sparse, distance-degenerate data in high dimensions	**Regularisation**	Constraints reducing overfitting
Data drift	Change in input distribution over time	**RLHF**	RL from human feedback
DPO	Direct Preference Optimisation for alignment	**RoPE**	Rotary positional embedding
Embedding	Dense vector representing an item/token	**SFT**	Supervised fine-tuning
Epoch	One full pass over training data	**Softmax**	Normalises logits into probabilities
F1 score	Harmonic mean of precision and recall	**Temperature**	Scales logits before sampling
Fine-tuning	Continued training on a task-specific dataset	**Tokenizer**	Maps text to integer tokens
GAN	Generator–discriminator adversarial model	**Transfer learning**	Reusing pretrained knowledge
Gradient	Vector of partial derivatives $\partial L/\partial \theta$	**Underfitting**	Model too simple for data
GQA	Grouped-query attention sharing K,V	**VAE**	Variational autoencoder
Hallucination	Fluent but unsupported model output	**Vector DB**	Store optimised for similarity search
HNSW	Hierarchical navigable small-world ANN index	**Warm-up**	Gradual LR increase at start
Inference	Using a trained model to predict	**Weight decay**	Penalty shrinking weights
Jacobian	Matrix of first-order partial derivatives	**XGBoost**	Regularised gradient-boosted trees
KV-cache	Stored keys/values to speed autoregressive decoding	**YaRN**	Method to extend RoPE context length
KL divergence	Asymmetric distance between distributions	**Zero-shot**	Performing a task with no examples
Learning rate	Step size of the optimiser	**ZeRO**	Zero Redundancy Optimizer state sharding
LoRA	Low-Rank Adaptation for PEFT		

16.4 Final 24-hour checklist

- Re-derive: linear/logistic regression gradient, backprop for a 2-layer MLP, softmax-CE gradient, attention with shapes.
- Prepare 3 STAR stories and a 2-minute crisp walkthrough of your best project (problem, data, model, metric, impact, what you'd change).
- Rehearse two system designs aloud (one classic ML, one LLM/RAG) with numbers: QPS, latency, storage, cost.

- Practise 3 coding problems in Python + 1 SQL window-function query; keep a note of shape checks and edge cases.
- Sleep well. Clarify assumptions early, think aloud, and structure answers with **ABCD**.

> **INTERVIEW TIP** **Good luck!** Depth beats breadth: knowing *why* each technique works — and where it breaks — is what distinguishes a strong AI/ML engineer.